Technical Reference

Firmware, GPIB Interface and Calibration System

Community Reverse-Engineering Supplement
Firmware, memory map, calibration data format, the complete GPIB command set including commands absent from the Operation Manual, and the calibration procedure performed over the bus

Document R6161-90T01

Jared Cabot • Licensed MIT

Revision 1.0 • July 2026

Scope

This document describes the internal operation of the Advantest R6161 DC Voltage/Current Generator as it relates to remote control and calibration: the GPIB command set including commands that do not appear in the Operation Manual, the firmware structure, the layout and encoding of the calibration data, the behaviour of the calibration procedure when performed over the bus, and a transport characteristic of the instrument's GPIB listener that governs how commands must be sequenced.

Provenance

The command set, handler addresses, access gating and memory map were obtained by disassembling the instrument's firmware, revision A05, from a 27C512 device image. Every statement about behaviour was then confirmed on a working instrument over GPIB, with a Keysight 3458A as the measurement reference.

Where a statement rests on firmware analysis alone, or on a single observation, the text says so. Chapter 8 records what was measured and the section "Not Verified" in Chapter 8 lists what remains unverified.

Applicability

Firmware revision A05. The instrument reports its revision in response to *IDN?. Other revisions are expected to be similar, but the addresses quoted here will not apply to them.

Conventions

Hexadecimal numbers are prefixed with a dollar sign, whether they are addresses or register contents. Instrument commands are set in a monospaced face. References of the form "manual 7.2.2" are to the R6161 Operation Manual, October 1991 printing.

Disclaimer

This document is not a product of Advantest Corporation and carries no endorsement from it. It records the findings of an independent examination of one instrument. The procedures described here can alter the instrument's calibration.

Contents

1. General Information **1-1**
Instrument Description 1-1
Ranges 1-1
Accuracy Specification 1-2

2. Remote Operation **2-1**
Interface Capability 2-1
Description 2-1
Measurements 2-2
Recommended Handling 2-3

3. Programming Reference **3-1**
Command Dispatch and Access Gating 3-1
Calibration Commands 3-2
Commands Available in Either State 3-2
Undocumented Commands Outside Calibration 3-3
Output Control and Limits 3-4

4. Firmware and Memory **4-1**
Memory Map 4-1
The Three Copies of the Calibration Data 4-2

5. Calibration Data Format **5-1**
Block Layout 5-1
The Third Array 5-2
What the Firmware Stores 5-4
Integrity 5-6

6. Calibration Over GPIB **6-1**
The Point Set 6-1
Command Sequence 6-1
The Acceptance Window 6-4
Properties of the Procedure 6-6
Connections 6-6
Feedback During Calibration 6-8
Recommended Validation 6-9
After the Commit 6-9

7. Error Codes **7-1**
GPIB Error Codes 7-1
Codes Absent from the Operation Manual 7-2
The Address Error 7-3
The Complete Code Map 7-4
The Status Byte 7-7

The Fan Message 7-9
Panel and Program Error Codes 7-10
BCD Remote Control Error Codes 7-10

Contents (cont'd)

8. Verification Record **8-1**

Coefficient Decode 8-1

Adjustment and Re-verification 8-1

Confirmed by Observation 8-2

Not Verified 8-4

9. Command Reference **9-1**

Introduction 9-1

Notation 9-1

Dispatch Index 9-1

Calibration Commands 9-5

Memory Access Commands 9-9

Output Control Commands 9-10

Sense and Guard Commands 9-13

Memory, Recall and Program Commands 9-14

Status and Service Request Commands 9-18

System Commands 9-20

TR6120A Mode Commands 9-22

1

General Information

Instrument Description

The R6161 is a programmable DC voltage and current source introduced in 1991. It generates seven voltage ranges and three current ranges, each to six digits, and reaches 1.2 times nominal full scale in overrange. Control is by front panel or by IEEE-488 (GPIB).

Internally the instrument is built around a Z80 processor whose firmware is held in a UV-erasable EPROM, battery-backed static RAM, and an EEPROM that holds the calibration data. Three of the ten ranges — 10 mV, 100 mV and 1000 mV — are produced by a resistive divider and emerge from a separate pair of output terminals. That distinction governs the whole of the calibration procedure and is described in Chapter 6.

Ranges

Table 1-1 lists the range codes used in normal operation, the corresponding calibration-point selector described in Chapter 3, and the terminals each range appears on. Full scale is quoted in the range's panel unit, which is also the unit in which its calibration data is entered.

Table 1-1. Ranges and Range Codes

Range	Select	Cal. Selector	Full Scale	Terminals	Resolution
10mV	V2	RV2	10 mV	Divider	1e-08
100mV	V3	RV3	100 mV	Divider	1e-07
1000mV	V9	RV9	1000 mV	Divider	1e-06
1V	V4	RV4	1000 mV	Main	1e-06
10V	V5	RV5	10 V	Main	1e-05
100V	V6	RV6	100 V	Main	0.0001
1000V	V7	RV7	1000 V	Main	0.001
1mA	I1	RI1	1 mA	Main	1e-09
10mA	I2	RI2	10 mA	Main	1e-08
100mA	I3	RI3	100 mA	Main	1e-07

Accuracy Specification

The Operation Manual quotes total accuracy as the sum of a term proportional to the setting and a fixed adder, as shown in Table 1-2. Total accuracy includes the traceability of the external standards, calibration error, stability, temperature coefficient, long-term stability, linearity and noise ripple from DC to 1 Hz. Line regulation and load regulation are excluded.

Table 1-2. Total Accuracy

Range	24 Hours	90 Days	180 Days
10mV	±(0.0055 % + 7e-07)	±(0.0060 % + 2.3e-06)	±(0.0065 % + 2.3e-06)
100mV	±(0.0040 % + 8e-07)	±(0.0045 % + 2.5e-06)	±(0.0050 % + 2.65e-06)
1000mV	±(0.0030 % + 6e-06)	±(0.0035 % + 8e-06)	±(0.0040 % + 8e-06)
1V	±(0.0020 % + 1e-05)	±(0.0025 % + 1.1e-05)	±(0.0030 % + 1.2e-05)
10V	±(0.0020 % + 6e-05)	±(0.0025 % + 7e-05)	±(0.0030 % + 7e-05)
100V	±(0.0020 % + 0.0006)	±(0.0025 % + 0.0007)	±(0.0030 % + 0.0007)
1000V	±(0.0025 % + 0.006)	±(0.0030 % + 0.007)	±(0.0035 % + 0.007)
1mA	±(0.0055 % + 9e-09)	±(0.0060 % + 9e-09)	±(0.0065 % + 9e-09)
10mA	±(0.0040 % + 9e-08)	±(0.0045 % + 9e-08)	±(0.0050 % + 9e-08)
100mA	±(0.0040 % + 9e-07)	±(0.0045 % + 9e-07)	±(0.0050 % + 9e-07)

The fixed adder is expressed in SI base units: volts for the voltage ranges and amperes for the current ranges. No one-year figure is published. Separate relative-accuracy tables appear in the Operation Manual; they exclude the traceability of the external standards and are not appropriate for certification.

Note



The specification is stated for the conditions in which it was measured: more than one hour of warm-up, an ambient of 23 ±5 °C and relative humidity of 70 %% or less. The current ranges are assured within an output compliance of ±10 V.

Initial Setup Status

The state the instrument adopts at power-up, and the state a program should leave it in, are the same thing, and the Operation Manual states it in its own Table 3-1 (section 3.1.3). Table 1-3 reproduces the part that matters to a program, together with the column that is easy to overlook: whether each parameter is **backed up**.

Table 1-3. Initial Setup Status, from Operation Manual Table 3-1

Parameter	Initial value	Backed up
Generation function	VOLTAGE	no
Range	1 V	no
Generation value	+0.000000 V	no
Voltage limit	130 V	no
Current limit	125 mA	no
AUTO RANGE	ON	no
OPR / STBY	STANDBY (output off)	no
EXT-GUARD	OFF	**yes**
EXT-SENSE	OFF	**yes**
GPIB mode / header / address	R6161, header on, addressable, 8	**yes**

The parameters marked as not backed up return to the values above at every power-up, so a program need not restore them: the function, the range, the generation value, both output limits and the standby state all reset themselves. Holding the `CE` key while switching on forces the whole table, which is also what the `C` command does.

Caution



Sense and guard are backed up, so whatever a program leaves them at survives the power cycle. They are the two parameters in the table that a calibration necessarily changes — manual 7.2.2(1) measures straight across the output terminals, which is internal sense — and they are the two that will not reset themselves afterwards.

The initial value of both is OFF, and that is what a program should leave them at. Restoring them to EXTERNAL instead, on the assumption that external was the operator's preference, leaves an instrument that regulates through leads which may no longer be connected, and nothing on the front panel explains why its output is wrong.

2

Remote Operation

This chapter describes a behaviour of the instrument's GPIB listener that must be accounted for by any program that controls it. It is not described in the Operation Manual, and the manual's own example programs do not accommodate it. It opens with the interface's declared capability, which bounds what a controller may reasonably expect of it.

Interface Capability

Table 2-1 reproduces the interface function subset declared in Operation Manual Table 4-1. Two entries matter to a calibration program: `C0` means the instrument is never a controller, so it cannot address itself to talk, and `DC1` means Selected Device Clear and Device Clear are both honoured — the recovery of last resort when the listener wedges.

Table 2-1. Interface Functions, from Manual Table 4-1

Code	Function
SH1	Source handshake, complete capability
AH1	Acceptor handshake, complete capability
T6	Basic talker; serial poll; talker released by MLA
L3	Basic listener; listener released by MTA; listen-only mode
SR1	Service request, complete capability
RL1	Remote/local, complete capability
PP0	No parallel poll
DC1	Device clear; SDC and DCL both accepted
DT1	Device trigger; GET accepted
C0	No controller capability
E2	Three-state bus drivers

Note



The device address and the R6161 / TR6120A command-set selection are both front-panel settings, reached by pressing LOCAL to enter GPIB Addressing mode; see manual 4.4 and the notes at the head of the TR6120A section of Chapter 9. Neither can be read or written over the bus, so a program cannot verify which command set the instrument is listening for.

Description

Any command that is immediately preceded by a write-only command loses its leading character. A command preceded by an operation that addresses the instrument to talk arrives intact. The lost character is not recovered; the truncated remainder is parsed as though it were the command sent.

Because the remainder is usually a different valid command, the symptom is not a communication failure but a plausible and misleading error. The four cases below were all observed.

```
SEN1   ->  EN1     ->  Err.G01   (no such command)

*IDN?  ->  IDN?    ->  Err.G07   (I handler; 'D' is not 1, 2 or 3)

*STB?  ->  STB?    ->  Err.G11   (ST handler; 'B' is not 0, 1 or 2)

RV4+0  ->  V4+0    ->  Err.G15   (V handler; gated to EXT CAL off)
```

The last case is the practical consequence. A calibration session consists almost entirely of write-only commands, so once the first command has been sent every subsequent one arrives truncated. `RV4+0` becomes `V4+0`, which reaches the ordinary range-select handler; that handler is gated to EXT CAL = DISABLE, so it raises Err.G15. An operator sees a stream of Err.G15 and concludes that the calibration commands are being rejected, when in fact they are not arriving.

Measurements

Measured by first-attempt success of a no-retry `*IDN?`, with no retries and no device clear. A truncated `*IDN?` cannot answer, so first-attempt success is an unambiguous indicator. Table 2-2 gives the result.

Table 2-2. Effect of Preceding Traffic

Preceding Traffic	Successful	Rate
Query only, nothing before it	25 / 25	100 %
One write-only command immediately before	2 / 25	8 %
One write-only command, then a 1.0 s pause	0 / 25	0 %
Two write-only commands before	2 / 25	8 %

The fault is not a timing problem. A one-second pause after the write made the result worse, not better, which excludes settling as an explanation and points to transaction state. The manual's own figures put that beyond doubt: Operation Manual A1.1.2 tabulates the execution time of every program code, and the slowest is `RCL` at 6.2 ms of handshake plus 42 ms of internal processing. Table 2-3 gives the extremes. A one-second pause is some twenty times the longest operation the instrument performs.

Table 2-3. Execution Time Extremes, from Manual A1.1.2

Program Code	Handshake	Internal Processing
RCL<nn>	6.2 ms	42 ms
VL<n>, IL<n>	4.1 ms	33 ms
MEM	24.5 ms	15.8 ms
D<value>	7.2 ms	12 ms
V<d>, I<d>	2.8 ms	3.3 ms
E, OP, H, SB	2.6 ms	2.4 ms
PANE? (query)	5.8 ms	26 ms to send
*IDN? (query)	5.4 ms	19 ms to send

No write terminator avoids the behaviour. All four block delimiters the Operation Manual permits in section 4.5.5 were measured, together with EOI suppression, against a baseline of 20 of 20. Table 2-4 gives the result.

Table 2-4. Write Terminator Survey

Write Terminator	Successful	Rate
LF (the usual choice)	2 / 20	10 %
CR LF	0 / 20	0 %
CR	2 / 20	10 %
EOI only, no characters	1 / 20	5 %
LF with EOI suppressed	1 / 20	5 %

Recommended Handling

Serial-poll the instrument after every write-only command. It is the act of addressing the instrument to talk that clears the condition, not any data transferred, and a serial poll is the cheapest operation that does so: it sends no command, so it can raise nothing on the front panel, and it returns immediately. Table 2-5 gives the measured result.

Table 2-5. Effect of an Interposed Talk-Addressing

Interposed	Next command intact	Next query answered	Cost each
Nothing	2 / 20	40 / 45	0.00 s
Serial poll	20 / 20	45 / 45	0.00 s
Bare read	20 / 20	45 / 45	2.04 s

The first column is the command-error bit after a second command sent back to back with the first, over four pairs including `SEN0` then `SEN1` and `V5` then `VL0130`. The second is a query attempted after each of nine different commands. Both columns give the same answer, which is expected: one condition, one remedy.

A bare read is equally effective and is the right fallback where a poll cannot be taken, but it costs about two seconds. It has nothing to read, so it always times out, and a fruitless read is not governed by the timeout requested of it — it costs whatever the GPIB driver imposes, measured at 2.04 s against a requested 600 ms, the same fixed cost as polling an address with no instrument on it. Over the 150 commands of an adjustment that is five minutes of a run spent proving nothing.

Note



Neither remedy may be a throwaway *query*. A query sends a command, which during calibration is itself refused and lights the front panel; a full adjustment would display roughly 120 spurious errors. A serial poll and a bare read both send nothing, and both work while every query is gated — the poll because it is served by the interface chip rather than the firmware, the read because a read that times out is as effective as one that succeeds. That is what makes them usable in the session they are most needed for.

One poll is required per command. A single poll does not cover a run of consecutive writes: two writes followed by one poll leaves the following command corrupted.

Caution



Take care when measuring any of this. A serial poll is the remedy, so a poll placed in a control sequence — to read the status byte between two commands, say — silently applies the remedy and makes the control identical to the treatment. Two attempts at Table 2-5 were invalidated that way, both appearing to show that back-to-back commands were unaffected. Read the status byte before the sequence under test, not inside it.

One command needs more than that. `RCL` applies a complete stored setup, and a single interposed talk-addressing does not reliably clear the state it leaves; these were measured with reads. Table 2-6 gives the measured result.

Table 2-6. Recovery After RCL

Handling	Following Command Answered	Rate
One interposed read, 0.3 s gap	17 / 20	85 %
One interposed read, 1.5 s gap	9 / 10	90 %
Two interposed reads, 0.3 s gap	29 / 30	97 %

The extra delay barely helps and the extra read mostly does. The gaps tested were six and thirty times `RCL`'s total execution time of about 48 ms, so the command had long since finished in every case. A residual few per cent survives even two reads, so a program that must be certain a recall took effect should read the setup back rather than assume it.

Note



That `RCL` is the worst offender is consistent with it being the slowest code in the instrument: its 42 ms of internal processing is three and a half times the next longest, `D` at 12 ms. How much work a command does appears to govern how badly it leaves the listener, even though waiting for that work to finish does not repair it. `RCL` also puts the instrument into program mode, where it stops answering altogether, so a program that must be certain a recall took effect should read the setup back rather than assume it.

The Operation Manual permits up to 400 characters per block with comma separators, and the manual's own examples use that form, for example `V6,VL50,IL20,SEN1,GRD1`. Combining commands into one block reduces the number of writes and is a useful secondary measure, though it does not remove the need for the poll.

3

Programming Reference

Command Dispatch and Access Gating

The command parser is a linear comparison chain rather than a lookup table. Each entry loads a pointer to an $FF-terminated mnemonic, calls the string comparator at $7B94, and jumps to the handler on a match.

```
LD   DE,<mnemonic>
CALL $7B94          ; compare against the input buffer at ($D612)
JP   z,<handler>
```

The mnemonic table begins at $9CC5 and holds 74 entries. Two dispatch chains exist, one for R6161 mode and a shorter one for TR6120A compatibility mode. Chapter 9 lists every entry.

Most handlers begin with a call to $93B0, followed by a conditional jump to an error exit. Two forms occur, and they are opposites.

```
CALL $93B0 / JP z,$9981     command requires EXT CAL = ENABLE
CALL $93B0 / JP nz,$9985    command requires EXT CAL = DISABLE
```

Note



$93B0 does not read the switch itself. It returns the cached flag at $DE5A, which the periodic scan maintains: $0438 samples the status port $FEC0 and dispatches on bit 3, and the routines at $2085 and $22C7 clear or set the flag accordingly. The distinction matters only in that the gate reflects the switch as of the last scan rather than at the instant the command is parsed.

A handler with no such call works in either state. This three-way division explains the behaviour of the instrument during calibration completely: the calibration commands work only with the switch enabled, nearly everything else works only with it disabled, and a small set — notably OPERATE and STANDBY — works in both, which is what allows a calibration procedure to energise the output at each point.

Calibration Commands

Table 3-1 lists the commands that work while EXT CAL is enabled. None of them appears in the Operation Manual.

Table 3-1. Calibration Command Set

Command	Handler	Function
RV<d>{+|-}{0|FS}	$80EC	Select a voltage calibration point. <d> is the range digit 2, 3, 4, 5, 6, 7 or 9. Takes no value.
RI<d>{+|-}{0|FS}	$809A	Select a current calibration point. <d> is 1, 2 or 3. Takes no value.
RVL	$8200	Select the voltage-limit calibration point. No suffix, no value.
RIL	$81E1	Select the current-limit calibration point. No suffix, no value.
RKTA	$8034	Select DAC connection point 1, the base point of the pair.
RKTB	$8052	Select DAC connection point 2, one coarse-register step above point 1.
RKTD	$806C	Select DAC connection point 3, one fine-register step above point 2.
RD<value>	$8D19	Enter the measured value for the selected point. Signed, up to seven digits, in the point's entry unit. The point must be in OPERATE and the value must lie within 1 % of nominal.
REND	$7F83	CAL END. Commits every entered coefficient to EEPROM. The only operation that makes a calibration change persist.

Note



Order matters. `RD` tests two state variables before accepting data — `$DE74` must be zero and `$DDE8` must be non-zero — both of which are set by a preceding point selection. An `RD` that does not follow a selector is discarded.

Commands Available in Either State

The handlers in Table 3-2 contain no gate call and work with the EXT CAL switch in either position. The presence of OPERATE and STANDBY here is what makes an automated calibration possible: each point must be energised before it can be measured.

Table 3-2. Ungated Handlers

Command	Handler	Function
E, OP	$7F5D	OPERATE. Energises the output terminals.
H, SB	$821F	STANDBY. De-energises the output terminals.
SEN0 / SEN1	$7E5E / $7E93	Internal / external sense.
GRD0 / GRD1	$7E09 / $7E33	Internal / external guard.
L<n>	$7EC9	TR6120A-mode limit selection, L1 to L8, per manual 4.5.6.

Undocumented Commands Outside Calibration

Two commands outside the calibration set are implemented in firmware but do not appear anywhere in the Operation Manual: neither in the R6161-mode program-code list of section 4.5.2, nor in the TR6120A-mode list of section 4.5.6, nor in the query table of section 4.6. Both are memory access commands, evidently left in the production firmware for service use. They are listed in Table 3-3.

Table 3-3. Commands Absent from the Operation Manual

Command	Handler	Function
MRB<addr>?	$8BEE	Read one byte of processor memory, address in hexadecimal. The route by which calibration data can be inspected.
MWB<addr>,<val>	$8C30	Write one byte of processor memory. Accepted, and the store at $8C85 executes, but see "The Three Copies of the Calibration Data" in Chapter 4.

Several commands that a reader might expect to be undocumented are in fact described in the Operation Manual, and are recorded here so that they are not mistaken for service commands. `*IDN?` appears as item (6) of manual Table 4-5; `PANE?` is the batch parameter read of section 4.6.1; `MEM` is the memory store of section 4.5.3; `RCL`, `SMS`, `*TST?` and `*CLS` are items (5), (11), (9) and (10) of manual Tables 4-3 and 4-5; and `L` is the TR6120A-mode limit command of manual Table 4-4. Chapter 9 gives the manual reference for every dispatched mnemonic.

Note

Manual Table 4-5 gives the `*IDN?` response as ADVANTEST, R6161, REV A01. The instrument examined returns REV A05, from the string at `$9DE3`. The revision field tracks the firmware, not the manual printing.

Note

The status-byte mask set by `SMS` powers up at 255, with every bit enabled, so a program that polls the status byte sees command errors without having to change anything. It is `SMS0` that hides them.

Warning

Never send `Z` or `*RST`. Both reach handler `$7FAE`. Operation Manual section 4.5.2(13) states that this initialises the battery-backed program data. Calibration coefficients are among the data affected, and there is no undo. Neither command should be used as a convenient invalid command when testing.

Output Control and Limits

Range is selected with `V<d>` or `I<d>` using the digits in Table 1-1. The output value is set with `D<sign><number><unit>`. The unit suffix selects auto-ranging; with the suffix omitted the range stays fixed and **the number is interpreted in volts**, or milliamperes on the current ranges, regardless of which range is selected. The Operation Manual's own example is `V6` followed by `D+30` for 30 V on the 100 V range.

Note



The `D` unit is not the calibration entry unit. Calibration data for the 1 V range is entered in millivolts, per manual 7.2.2, but `D+1000` on the 1 V range means 1000 V and is rejected with Err.G13. The correct command for full scale on that range is `D+1`.

`VL<n>` sets the voltage limit, 10 to 1250 V in 10 V steps; `IL<n>` sets the current limit, 1 to 125 mA. Both handlers are gated to EXT CAL = DISABLE, so a limit can only be changed with the switch in that position. Neither can be set in Divider mode at all, which raises Err.G16, so a non-Divider range must be selected first. The initial values are VL130 and IL125.

Caution



At the initial VL130 setting the 1000 V range produces no output at all when it is driven by the `D` data command. The failure is silent and misleading: range selection succeeds, OPERATE succeeds, and the range's zero point still measures correctly, so only a full-scale measurement reveals it. Any procedure that sets the 1000 V range's output with `D` must raise the limit first.

The limit does not affect the calibration procedure, where the firmware drives the output from the selected point rather than through `D`.

4

Firmware and Memory

Memory Map

Table 4-1 lists the addresses that concern calibration.

Table 4-1. Memory Map

Address	Contents
$0000 - $9FFF	Firmware in UV-EPROM: executable code and constant tables.
$9AE9	Factory-default calibration data, 364 bytes. Used when the EEPROM contents fail their integrity check.
$9CC5	GPIB mnemonic table, 74 $FF-terminated entries.
$9DE3	The *IDN? response string.
$D612	Received-command buffer, read by the parser.
$DC29	Working calibration data, 364 bytes, in battery-backed RAM. Overwritten from the EEPROM master at every boot.
$DDE8, $DE74	Calibration state; tested by RD.
$DE2A - $DE33	DAC connection point selection slots, set by RKTA, RKTB and RKTD.
$DE3F, $DE40	Voltage-limit and current-limit point flags, set by RVL and RIL.
$E000	Calibration master in EEPROM, 364 bytes.
$E16C	Checksum of the master block: the low eight bits of the sum of its 364 bytes.
$FEC0	Status input port, sampled at $0438 and cached at $D685. Table 4-2 gives the bits.
$21A2	The EEPROM commit routine, reached from REND.

The status port carries more than the calibration switch. `$0438` samples it once per pass, caches the value at `$D685`, and dispatches on the individual bits; Table 4-2 lists them. On the instrument examined the port read `$4F` with the switch disabled, the fan running and no fault present.

Table 4-2. Status Input Port $FEC0

Bit	Sense	Meaning
1	low	Fault input; sets $C004. Gates an output at $FE40 together with bit 2, and is one of the two conditions behind the limit bit of the status byte.
2	low	Fault input; sets $C003. The companion of bit 1.
3	low	Rear-panel EXT CAL switch. The result is cached at $DE5A, which is what the gate routine $93B0 reads.
4	high	Fan fault. Sets $C00B and shows the panel message at $9AA7; see Chapter 7.
5	high	Digit-adjust strobe. Applies one increment or decrement to the display digit selected by $DDFA; see below.
6	—	Direction for bit 5. Set counts up, clear counts down.
7	high	ENTER strobe. Writes the key code 5 to $D67C, raises the pending-action flag $DDDB and calls the entry routine at $0466.

The Digit-Adjust Path

Bits 5, 6 and 7 together are the front panel's data entry. The handler for bit 5 is at `$2352`. It reads `$DDFA`, which holds the digit position as 1 to 7, ignores the strobe if that byte is zero or 8 or more, and otherwise dispatches through `$045D` into one of two seven-entry jump tables: `$0043` when bit 6 is set and `$0051` when it is clear. The two tables are the increment and decrement routines for the seven digits of the display buffer at `$D68E` to `$D694`, each carrying into the next digit on wrap.

The handler then clears bit 5 in the cached copy at `$D685` and re-enters the bit chain at `$043E`, so several strobes present in one sample are serviced in turn. Bit 7 does the same for itself. This is the mechanism behind the observation in Chapter 5 that the DAC coefficient can be recomputed by a key press: the ENTER strobe reaches the key handlers that clear `$DE45`.

The Three Copies of the Calibration Data

The calibration data exists in three places: the factory defaults in ROM, the master in EEPROM, and the working copy in battery-backed RAM. At power-up the master is copied over the working copy. All measurement corrections are applied from the working copy.

This arrangement has a consequence that is easy to misread. Writing the working copy with `MWB` does change the instrument's behaviour, but the change is lost at the next power-up. Writing the master's address range has no effect at all: the store instruction executes and the syntax-error bit stays clear, but the value does not change, because an EEPROM requires a programming sequence rather than a processor store. Only the commit routine reached by `REND` writes the master.

Note



A calibration image can therefore be read for record or comparison, but cannot be written back. There is no restore operation. If a calibration is lost, the values must be re-entered through the normal procedure.

5

Calibration Data Format

Block Layout

The calibration block is 364 bytes and contains three arrays, described in Table 5-1. All numeric values are little-endian packed BCD: two decimal digits per byte, least-significant byte first.

Table 5-1. Calibration Block Layout

Offset	Address	Records	Size	Contents
$000	$DC29	20	8 bytes	Full-scale coefficients. Value = BCD / 10^6, in the range's panel unit.
$0A0	$DCC9	20	9 bytes	Zero coefficients. Value = BCD / 10^{10}, as a fraction of full scale. Eight BCD bytes followed by a flag byte.
$154	$DD7D	3	8 bytes	The DAC connection coefficient, then the current-limit and voltage-limit coefficients. Only the low four bytes of each record are used.

The three regions account for the block exactly: 160 plus 180 plus 24 is 364 bytes. Within both the full-scale and the zero array the first ten records are the positive points and the next ten the negative points, each in the order below.

```
1000V, 100V, 10V, 1V, 1000mV, 100mV, 10mV, 100mA, 10mA, 1mA
```

The Third Array

The third array does not hold three DAC coefficients, as its position after the two range arrays might suggest. It holds one DAC connection coefficient followed by the two limit coefficients, as set out in Table 5-2. Only the low four bytes of each eight-byte record carry data; the upper four are zero.

Table 5-2. The Third Array

Address	Entered By	Contents	Scaling	Unit
$DD7D	RKTA/RKTB/RKTD	DAC connection coefficient	BCD / 10^6	dimensionless
$DD85	RIL	Current limit	BCD / 100	mA
$DD8D	RVL	Voltage limit	BCD / 100	V

Which record is which was established three times over. In firmware, `$43EB` tests the voltage-limit flag at `$DE3F` and writes `$DD8D`, while `$4401` tests the current-limit flag at `$DE40` and writes `$DD85`. On the bench, each of the three was entered on its own and only the expected record moved.

The factory defaults in ROM settle it independently. Decoding the block at `$9AE9` with the layout above gives a DAC coefficient of 0.010000, a current limit of 50.00 mA and a voltage limit of 500.00 V — the nominal value of each quantity. No other assignment of the three records produces three sensible nominals.

The DAC Connection Coefficient

The three RKT points are not stored individually. They are three measurements of the same output at three DAC settings, and the firmware reduces them to a single ratio. The arithmetic is at `$4302` to `$4390`: it forms the difference between the second and first points, divides it by 10000, divides that by the difference between the third and second points, and stores the result scaled by 10^6. In terms of the entered values that is

```
stored  =  100 x (B - A) / (D - B)          value = BCD / 10^6
```

where A, B and D are the values entered at `RKTA`, `RKTB` and `RKTD`. The drive patterns confirm the roles: A sets both the coarse and fine registers to `$1050`, B advances the coarse register to `$1150`, and D then advances the fine register to `$7050`. A is therefore the base point, B differs from it by one coarse step and D from B by one fine step, so the stored value expresses the coarse step in units of the fine step.

The relation was confirmed by entering four deliberately chosen sets and reading the record back. Table 5-3 gives the result.

Table 5-3. Confirmation of the Coefficient Relation

A	B	D	Computed	Stored	Bytes
0.010000	0.110000	0.111000	10000.0	10000	00 00 01 00
0.010000	0.110000	0.111250	8000.0	8000	00 80 00 00
0.010000	0.115000	0.116000	10500.0	10500	00 05 01 00
0.010000	0.109789	0.110789	9978.9	9979	79 99 00 00

Note



The last set is the informative one: it computes to 9978.9 and stores 9979, so the coefficient is **rounded**. The two limit coefficients in the same array are **truncated** — 49555.60 stored as 49555 and 4855.56 as 4855. The paths are separate in firmware and the behaviours genuinely differ.

The Arithmetic Is Exact

The relation above is written in decimal for readability, but the firmware does not work in decimal and does not round in the ordinary sense. The entry buffers at $C022, $C02A and $C032 are eight-byte little-endian **integers** holding the entered seven-digit mantissa; the upper four bytes are zero and there is no exponent field. Read from the instrument after the 8000 set of Table 5-3 they hold 10000, 110000 and 111250.

Stage one leaves |B - A| scaled by 10^4 at $C0A5, stage two leaves |D - B| in the accumulator, and stage three divides the first by the second. Reading $C0A5 after the same set gives 1 000 000 000, which is $(110000 - 10000) \times 10^4$ exactly. The whole computation is therefore integer arithmetic:

```
C0A5   = |B - A| x 10^4

stored = floor( ( floor(C0A5 / |D - B|) + 50 ) / 100 )
```

Every entry in Table 5-3 follows from that expression with no residual, and so does the coefficient produced by the instrument's own last real adjustment: 100084×10^4 divided by 1003 is 997846, and 997846 plus 50, with the last two digits dropped, is 9978.

Note



$C0A5 is referenced twice in the whole of ROM — written at $4335 and read at $4365 — and it lives in battery-backed memory, so it survives a power cycle. Nothing else in the firmware can disturb it, which is what makes the computation reproducible to the digit.

When the Coefficient Is Computed

The three stages are not run as each point is entered. They are called as a unit from $0C5E, under two conditions: $DE28 must be set, which happens when the point sequence advances past RKTD, and $DE45 must be clear. The computation sets $DE45 when it finishes. A routine at $9913 clears both flags, along with the rest of the calibration state.

It does not follow that the coefficient is computed once. $DE45 is written at four sites, and two of them clear it: $3570 and $3627. Both sit in front-panel key handlers, and both are reached only by the branch that tests $DE5A, the cached EXT CAL flag — that is, only while the switch is enabled. Each therefore re-arms the computation, which then runs again on the next pass with $DE28 still set. **The coefficient is recomputed on those key events, from whatever the entry buffers hold at that moment, whether or not any** RKT **point has been re-entered.**

The two differ in what they set afterwards. $3627 sets $DE35, which recomputes and stops there. $3570 sets $DE34, which recomputes and then takes the branch at $0C85 described below.

Once Over the Bus, Repeatedly From the Panel

The fourth writer of $DE45 is at $992B, in a block that the calibration point selectors reach: RKTA, RKTB, RKTD and the four entry points of the RV handler all call $98E6, which falls through into it. That block clears $DE45 **and** $DE28 together, so selecting a point resets the sequence rather than re-arming the computation.

The distinction decides the behaviour of an automated adjustment. A program can only ever clear $DE45 by selecting a point, which closes the gate at the same time; the two panel handlers clear it while leaving $DE28 set. So:

- **Over the bus the coefficient is computed exactly once** per completed sequence, when the points advance past RKTD. No command can provoke a second computation without first resetting the sequence.

- **From the panel it is recomputed on every one of those two key presses** while EXT CAL is enabled, from whatever the entry buffers hold at that moment.

Note



The key codes are read from port `$F400` into `$D67C` at `$2EC3`, and the ENTER strobe writes the same byte at `$23AA`. Those are its only two writers, so a program cannot synthesise a key press; the panel path is genuinely unreachable from the bus.

The Coefficient Has Its Own Commit

Chapter 6 states that `REND` is what commits an adjustment. That is true of a session driven over the bus, but it is not the whole picture. When `$DE34` is set, the path at `$0C85` clears it, then copies eight bytes from `$DD7D` in the working copy to `$E154` in the EEPROM master through `$40A5`, and calls `$402E` to recompute the master checksum. The DAC connection coefficient alone is written to durable storage, immediately, with no `REND` anywhere in the path.

Caution



A front-panel DAC connection adjustment is committed the moment the key is pressed. There is no working-copy stage to retreat from and a power cycle will not undo it, because the master itself has been rewritten and re-certified. The escape described in Chapter 6 — that a run which stops short of `REND` commits nothing — applies to a GPIB session, not to the panel.

Note



`$DE34` is set at exactly one site, `$3577`, inside the front-panel key handler. Every other write to it, including the one in the GPIB command path at `$8D75`, stores zero. No bus command can reach the immediate commit, so for a program `REND` remains the only way to make a change durable.

The rounding is done by addition rather than by a rounding instruction. `$7192` clears the eight-byte addend, `$4374` writes 50 into its low byte, `$6EF8` adds it to the accumulator, and the record is then taken from `$C056` — one byte above the accumulator base, so the low byte is discarded. Adding 50 before discarding is round-half-up.

The accumulator is binary throughout. `$72B3` converts it to BCD by double dabble, and the record is then taken from `$C056` rather than `$C055`, which discards the lowest BCD byte and with it two decimal digits. The stored figure is therefore *floor*((quotient × 100 + 50) / 100).

Note



That the addend is cleared first matters. It rules out the obvious explanation for the anomaly recorded in Chapter 8, which is that the rounding step might pick up stale bytes from an earlier calculation: it cannot, because the whole operand is zeroed immediately before the constant is written into it.

What the Firmware Stores

The value written to a full-scale record is not the number entered. The firmware stores **the measured full scale minus the measured zero** for that polarity. This was established by comparing the values entered during an adjustment against the records read back afterwards; on the 1 V range the computed difference was 1.00111814 and the stored record was 1.001118.

Zero records track the entered value to about one part per million of full scale, the small residual being the firmware's own rounding.

The 10 V positive full-scale record at `$DC39` read as follows on an instrument in adjustment. The byte sequences below are generated by the same codec the tool uses, so they cannot disagree with it.

```
bytes:   50 78 00 10 00 00 00 00

digits:  50 + 78x10^2 + 00x10^4 + 10x10^6  =  10 007 850

value:   10 007 850 / 10^6  =  10.007850   (panel unit: V)
```

The corresponding zero record at $DCDB:

```
bytes:   20 00 71 12 00 00 00 00   flag 01

digits:  12 710 020

value:   12 710 020 / 10^10  =  0.001271002  (fraction of 10 V)

         i.e. an offset of 12.71 mV on the 10 V range
```

The Unit a Record Is Stored In

A full-scale record holds the measured value in the record's own unit, which is usually — but not always — the unit the point is entered in. Table 5-4 gives the master block of an instrument in adjustment, read on 26 July 2026.

Table 5-4. Full-Scale Records of an Instrument in Adjustment

Range	Stored	Entered in	Stored in
1000 V	1000.706147	V	V
100 V	100.063307	V	V
10 V	10.007850	V	V
1 V	**1.001113**	**mV**	**V**
1000 mV	1000.769990	mV	mV
100 mV	100.071350	mV	mV
10 mV	10.008320	mV	mV
100 mA	100.132549	mA	mA
10 mA	10.011873	mA	mA
1 mA	1.000784	mA	mA

Caution



The 1 V range is the exception. It is entered in millivolts — RD takes 1000.000 there, and 1010.000 is the edge of the acceptance window — but its coefficient is stored in volts. Every other range stores in the same unit it is entered in.

Every record above sits within 0.14 % of its nominal, so the 1 V range's three-decade offset is unmistakable rather than a matter of interpretation. Software that checks a record against its nominal will report a false failure on this one range unless the storage unit is taken from a table rather than from the entry unit.

Integrity

The master block is followed by a single checksum byte at $E16C, holding the low eight bits of the sum of the 364 bytes before it. The EEPROM region is therefore 365 bytes, not 364.

The byte is maintained by the commit itself. REND reaches a routine at $21A2 that copies the whole working block to the master, and then one at $402E that re-sums what it just wrote and stores the result at $E16C. Two checks read it back: the power-up sequence at $0241 and the *TST? handler at $8815, which sets bit 3 of the self-test byte on a mismatch.

If the check fails at power-up the instrument displays an error for about five seconds, substitutes the ROM defaults at $9AE9, and continues; it will then operate but will be out of adjustment. The displayed code is described in Chapter 7.

The working copy and the master normally agree, so comparing them is a cheap check that a commit completed. On the instrument examined the master summed to `$96` and `$E16C` read `$96`.

Caution



The checksum is recomputed from whatever the working copy contains at the moment of the commit. It certifies that the block was stored intact, not that its contents are correct. A coefficient left at zero by an incomplete adjustment is committed as valid data, and neither the power-up check nor `*TST?` will report anything wrong.

6

Calibration Over GPIB

The Point Set

The procedure follows Operation Manual section 7.2.2 exactly; only the means of operating it differ. It comprises 45 points, listed in Table 6-1.

Table 6-1. The Complete Adjustment

Points	Selector	Description	Entry Unit
3	RKTA, RKTB, RKTD	Three measurements on the 10 V range that together yield one stored coefficient	V
40	RV<d>, RI<d>	Ten ranges, four points each: +Zero, +F.S, -Zero, -F.S	the range's panel unit
1	RVL	Voltage limit, approximately 500 V; the entry must lie between 400 and 600 V	V
1	RIL	Current limit, measured as the drop across a 10 Ω standard and entered as the calculated current, 40 to 60 mA	mA

Command Sequence

```
(operator)  set EXT CAL to ENABLE     - physical switch, port $FEC0
  repeat for each of the 45 points:
    <selector>                        select the point
    E                                 OPERATE  - required, see below
    (settle, then measure on the reference standard)
    RD<measured value>                enter it NOW, still energised
  REND                                CAL END - commits to EEPROM
(operator)  set EXT CAL to DISABLE
```

Each point must be energised with `E` before it is measured, and left to settle before it is read. The manual gives three figures for that, in three different places, and only the first is where anyone would look for it.

Table 6-2. Settling Required Before a Reading

Ranges	Settling	Where the manual says so
all except those below	approximately 5 s	page 7-5, *before the Calibration Procedure*, note 8
1000 V	approximately 20 s	page 7-15, footnote *1 against the 1000 V steps
100 mA	approximately 60 s	pages 7-27 and 7-28, in the 100 mA steps themselves

Note 8 reads "Set the settling time of approximately 5 seconds after output generation (except for the 1000 V range)". The exception is not left open — it is answered twenty pages later, against the 1000 V range's own steps, as "Use the data displayed approximately 20 seconds after the OPERATE operation". A procedure written from note 8 alone will use a settling time for the 1000 V range that is a quarter of what the manufacturer asks for.

The third figure is buried deeper still. Pages 7-27 and 7-28 — the 100 mA range's own pages, and no others in the chapter — carry the line "The data about 60 seconds after the operation is follows". Twelve times the general figure, with no reason given and no cross reference from note 8. The likely reason is heat: 100 mA into the 10 Ω standard the same page specifies is 0.1 W, where every other point in the procedure dissipates a thousandth of that. It is followed here on the manufacturer's authority; unlike the other two it has not been checked by measurement.

Measurement supports the first two. Stepping each range from STANDBY to full scale and watching the output on an 8½-digit reference, the 1 V and 10 V ranges reach 10 ppm of full scale in about 0.7 s and the 100 V range in 1.5 s, but the 1000 V range needs 4.9 s for 10 ppm and about 9 s for 3 ppm. At eight seconds it is still 4 ppm low.

Caution



While a range is still settling its error is always negative. The output approaches its final value from below and never overshoots — on every range, in every repeat measured. An inadequate settling time is therefore a systematic bias rather than scatter: averaging more readings does not remove it, and during an adjustment it is written straight into the stored coefficient, where it becomes a permanent negative offset on that range.

Note 7 on the same page asks for an integration time of 20 PLC or more on the reference voltmeter.

Note



The settling time is a measurement requirement, not a firmware one. An entry made with no deliberate delay after `E` was accepted, while the same entry with the point selected but not energised was discarded. Shortening the delay therefore costs accuracy, not acceptance.

"No deliberate delay" is not the same as none at all. The controller used interposes its own command gap between commands, so about half a second still elapsed between `E` reaching the instrument and `RD` following it. What was shown is that no *additional* wait is needed; manual A1.1.2 allows OPERATE 2.4 ms of internal processing, so the margin is large, but the experiment did not probe the millisecond region.

Caution



A range's zero point and its full-scale point must both be entered. Entering a full-scale point on its own leaves that range's coefficient at **zero** in the working copy, and nothing reports it: the entry is accepted, no error is raised, and the record reads all zeros afterwards. This was observed on the 1 V range.

It does not stop at the working copy. `REND` commits the whole block and recomputes the checksum over it, so a zeroed coefficient is written to EEPROM as valid data and is reported as sound by both the power-up check and `*TST?`. This is the concrete reason behind the instruction in manual 7.2.2 that the two are calibrated as a pair, and it is why a run must never be committed part-way through a range.

Note



Re-issue the point-selecting command before every `RD`. Every point-selecting command begins by calling `$98E6`, which wipes `$DE09` to `$DE58`, so re-selecting is always safe and costs nothing. It is *not* sufficient on its own: see the caution below.

The reason originally given for this was that `$DE3B` — "a zero point has already been seen" — is not cleared on the success path, so a second `RD` after a zero point would set `$DE3C` and apply the *full-scale* band to it. **That did not happen.** Tested on 29 July 2026, a second `RD+5.000000` sent at the 1 V zero point with no intervening selection was accepted, where the reading of the firmware predicted a refusal. Either the flag is cleared somewhere that was missed, or the precondition is narrower than described. The advice stands on its own merits; the mechanism behind it does not.

Caution



Selecting a point takes it out of OPERATE, so a re-selection must be followed by `E` before `RD`. This is the single most expensive thing in this chapter to get wrong, because nothing reports it.

A procedure that measured every point of a range and then went back to enter them — re-selecting each point and sending `RD` without re-energising it — had all 45 of its entries accepted on the status byte and stored none of them. `REND` then committed a working copy that had never changed, and the read-back afterwards found a perfectly valid, correctly checksummed block, because that block was the calibration the instrument already held. Nothing anywhere reported a fault.

Demonstrated directly on 29 July 2026: eight entries at otherwise identical conditions, four sent after `E` and four after a bare selection. All eight were accepted; the four made in OPERATE moved the working copy and the four made in standby moved nothing.

Note



The safest arrangement is to enter each point immediately after measuring it, while it is still selected and still energised, rather than measuring a group and entering it afterwards. There is then no re-selection to forget and no state to re-establish.

Traffic to the reference standard in between does no harm. Confirmed on 29 July 2026: an `RD` still stored after the controller had addressed a 3458A and completed a full integration cycle on it between the measurement and the entry.

The Acceptance Window

The firmware refuses an entry that is implausible for the point selected, raising Err.G19 over the bus and P04 from the panel. There is not one rule but three, one per family of point, and they share nothing beyond the code they raise. Table 6-3 gives all of them.

Table 6-3. What RD Accepts, by Point

Point	Accepted entry	Firmware
RV<d>±F.S, RI<d>±F.S	0.99 N to 1.01 N inclusive, where N is the range's nominal full scale in its own display unit; the sign must match the point's polarity	$5495, pairs at $9C85+
RV<d>±0, RI<d>±0	magnitude not exceeding 0.01 N, either sign	$5469, limits at $9C65+
RKTA	magnitude not exceeding 0.100000 V, either sign	$53E6, limit at $9C55
RKTB	positive, greater than the RKTA value just entered, magnitude not exceeding 0.200000 V	$5406, limit at $9C5D
RKTD	positive, greater than the RKTB value just entered, magnitude not exceeding 0.200000 V	$543F, limit at $9C5D
RVL	400.0000 to 600.0000 V inclusive, positive	$3FF7
RIL	40.00000 to 60.00000 mA inclusive, positive	$3FF7

N is taken from a ROM constant chosen by which range flag is armed, not from anything measured or stored. `$52E8` maps the armed flag to an index of 0 to 3 and that index selects one of four pairs; the decoded values are 0.99–1.01, 9.9–10.1, 99–101 and 990–1010, with the corresponding one-sided zero limits 0.01, 0.1, 1 and 10. In every case N equals the range's full scale as printed on the panel, so the rule can be stated simply as **±1 % of full scale about the point's nominal**.

Note



The band was also measured directly, by sweeping each edge to the last digit on 27 July 2026. On the 1 V range 1010.000 mV was accepted and 1010.001 refused, 990.000 accepted and 989.999 refused; its zero point accepted 10.00000 mV and refused 10.00001. The 10 mV divider zero accepted 0.100000 mV and refused 0.100001, and the 1 mA full scale accepted 1.010000 mA and refused 1.010001. All ten edges are reproduced exactly by the constants above.

Caution



The DAC points do not use the ±1 % band, and assuming they do makes the procedure impossible. `$52E8` tests the three DAC flags before it reaches the range-point code and returns without ever consulting `$5495`. On a working instrument the measured outputs at the three sub-steps were 0.0127 V, 0.1128 V and 0.1138 V; a ±0.1 V band about zero — which is what the RKTA limit is, and which is easily mistaken for a general rule — would refuse the second and third of those, and no adjustment could ever be completed.

What the firmware actually requires is that the three increase: RKTB above RKTA and RKTD above RKTB, each under 0.2 V. The comparisons are against the values entered moments earlier in the same sequence, held at `$C03A` and `$C042`, which is why the three must be entered in the order A, B, D and in one pass.

The two limit points are wider still: ±20 % about 500 V and 50 mA rather than ±1 %, which is consistent with the manual describing those outputs only as approximate. Both windows were confirmed at both edges on 29 July 2026: `RVL` refused 380.0000, accepted 400.0000 and 500.0000 and refused 620.0000, while `RIL` refused 38.00000, accepted 50.00000 and 60.00000 and refused 62.00000.

Note



A further constraint was read out of `$3F9A` — that the entry must be *written* with exactly three integer digits for `RVL` and two for `RIL` — and the instrument does not enforce it. `RD+0500.000` and `RD+050.0000` were both accepted. Leading zeros evidently do not survive to reach `$C011`. The rule cannot be separated from the window test by any other entry, so what remains of it is unknown rather than disproved, and no procedure should depend on it.

Note



Unit suffixes are refused outright during calibration. `$94F6` and `$9544` send `V`, `MV` and `MA` to a syntax error whenever `$DE5A` is set, so an entry must be sign, digits and decimal point only, expressed in the point's own unit. The parser also truncates silently after seven digit characters.

Properties of the Procedure

Four properties of the procedure are worth relying on when automating it.

- **Nothing is committed until CAL END.** Entries accumulate in RAM. Abandoning a run at any point, for any reason, leaves the stored calibration exactly as it was. This makes the procedure safe to attempt and safe to interrupt.
- **The procedure is measurement-driven.** Each step selects a point, the output is measured, and the measurement is entered. No step needs to know what the point should have produced.
- **Zero and full scale are a pair.** Manual 7.2.2 states that re-calibrating one requires re-calibrating the other.
- **The DAC connection points come first,** because the range coefficients are measured through the DAC that they describe. The three of them yield one stored coefficient, not three; see Chapter 5.

Connections

The ten ranges cannot be presented on one pair of terminals, so the procedure requires the four separate connections in Table 6-4.

Table 6-4. Connection Groups

Group	Connection	Points
Main output	+OUTPUT and -OUTPUT to the reference voltmeter	3 DAC, 1 V, 10 V, 100 V, 1000 V, voltage limit
Divider output	the separate divider output terminals, Hi and Lo	10 mV, 100 mV, 1000 mV
Current	the reference ammeter, in series	1 mA, 10 mA, 100 mA
Current limit	a 10 Ω standard across the output, voltmeter across it	current limit

Note



Manual 7.2.2 directs that copper clips be used on the divider output terminals and that one minute be allowed after connecting for thermoelectric EMF to settle. At 10 mV full scale a thermal offset of 1 µV is 100 ppm, which exceeds the quantity being measured. Measuring the divider ranges on the main output terminals reads approximately zero.

Measuring the Current Ranges: Two Methods

The manual gives two ways to measure the three current ranges and does not say which is better. Steps (11), (13) and (15) describe the **voltage-drop** method — a standard resistor across the output terminals, the voltmeter reading the drop across it, the current obtained by dividing — while the caution on page 7-22 offers instead: "Connect a Digital current meter between the +OUTPUT and -OUTPUT terminals if a highly accurate one is available."

The choice is **per range**, not per procedure. Each range's resistor is a separate hookup and a separate step, so a bench holding one standard can use it on the range it suits and measure the other two directly; nothing in the procedure couples the three. Table 6-6 shows why that matters — the advantage of the drop method is not uniform, and on one range it is decisive.

The manual specifies a different resistor for each range, and the reason only becomes obvious when the three are listed together: every one of them puts **exactly 1 V** across itself at that range's full scale.

Table 6-5. Standard Resistors for the Voltage-Drop Method

Range	Standard	Drop at F.S	Manual
1 mA	1 kΩ	1 V	7.2.2(11), page 7-22
10 mA	100 Ω	1 V	7.2.2(13), page 7-25
100 mA	10 Ω	1 V	7.2.2(15), page 7-27

That is a measurement decision, not a convenience. It lands every reading at the top of the reference voltmeter's 1 V range, where the ppm-of-range term of its specification is smallest. Using one 10 Ω standard for all three ranges — the obvious simplification — would read 10 mV at 1 mA, a tenth of the way up the 100 mV range, and would be *worse* than measuring the current directly.

Table 6-6. Reference Uncertainty of Each Method, 3458A Example

Range	Direct DCI	Drop: volts	Drop: ohms	Drop total	R6161 90-day	Ratio direct	Ratio drop
1 mA	26.0	8.4	10.6	**13.5**	69	2.7:1	**5.1:1**
10 mA	26.0	8.4	18.0	**19.8**	54	2.1:1	**2.7:1**
100 mA	41.0	8.4	21.0	**22.6**	54	1.3:1	**2.4:1**

All figures are ppm of reading at that range's full scale, from the 3458A's one-year specification, with each term evaluated against the range's true full scale — every 3458A range is 20 % over-range, so the ppm-of-range term is multiplied by 1.2 and not by 1. The two voltage-drop terms are independent measurements of different quantities and are combined in quadrature; the worst-case linear sums are 19.0, 26.4, 29.4 ppm respectively. The last two columns are the test uncertainty ratio: the instrument's own specification divided by the uncertainty of the reference measuring it.

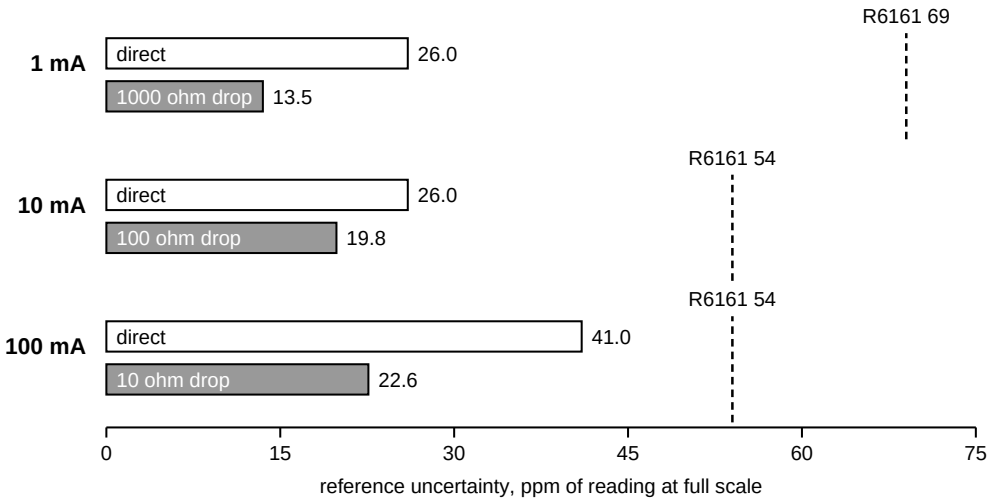


Figure 6-1. Reference uncertainty by method, with the R6161's own 90-day specification shown dashed. The voltage-drop method wins on every range, and by most on the 100 mA range, where the 3458A's current function is weakest.

The voltage-drop method is better on all three ranges and decisively better on 100 mA, where measuring directly gives a test uncertainty ratio of 1.3:1 — a reference barely better than the instrument it is grading. The usual target is 4:1, which nothing here reaches; the resistance measurement is what stands in the way, contributing more than the voltage reading on every range. A standard resistor with its own calibration certificate at a few ppm, rather than one measured on the same meter, would take all three well past 4:1.

A test uncertainty ratio is an abstraction until conformity has to be decided. Where it is decided with a **guard band** — conformity asserted only when the deviation is inside the specification by at least the reference's expanded uncertainty, so that a result too close to the limit to resolve is not claimed as a pass — the ratio becomes the fraction of the specification that remains usable:

Table 6-7. Specification Remaining After a Guard Band

Range	R6161 90-day	Direct U	Direct band	Drop U	Drop band
1 mA	69	26.0	43 (62 %)	13.5	**56 (80 %)**
10 mA	54	26.0	28 (52 %)	19.8	**34 (63 %)**
100 mA	54	41.0	13 (24 %)	22.6	**31 (58 %)**

All in ppm of reading at full scale. On 100 mA measured directly, three quarters of the instrument's own specification is consumed by the uncertainty of the reference and 13 ppm is left to hold the instrument to; the drop method leaves 31 ppm. Whether a guard band is applied at all is a matter for the issuing laboratory — simple acceptance, which ignores the uncertainty and grades against the specification alone, is equally legitimate and is what a manufacturer's verification normally does (ILAC-G8 section 4.2.1). The figures above are what the choice costs.

Caution



The standard resistor is characterised cold and used hot. The 3458A's four-wire ohms test current on its 10 Ω range is 10 mA, so a 10 Ω standard is measured dissipating about 1 mW — and then carries 100 mA in service, dissipating 0.1 W, a hundred times more. A resistor with a power coefficient of a few ppm per watt is fine; a wirewound of unknown provenance may not be. The 100 Ω and 1 kΩ standards are far less exposed, dissipating 10 mW and 1 mW in service.

This is the most likely explanation for the manual asking, on the 100 mA range and on no other, for "the data about 60 seconds after the operation". It is the only point in the whole procedure where anything gets warm.

Feedback During Calibration

Every query is refused while EXT CAL is ENABLE, so an automated procedure appears to be working blind. It is not. The IEEE-488 serial poll is answered by the GPIB controller rather than by the command dispatcher, and none of the routines that carry an error into the status byte is gated, so the poll continues to work throughout a run.

Bit 1 of the poll response reports whether the most recently executed command line raised an error, and it is cleared by the next line that completes cleanly. Because a point-selecting command precedes every `RD` and is itself a clean line, a poll taken immediately after the `RD` describes that entry and nothing else. The mechanism, its firmware path and the bench measurements confirming it are given under "The Status Byte" in Chapter 7.

```
<selector>          clean line       ->  bit 1 cleared

E                   clean line

(settle, measure on the reference standard)

RD<measured value>                   ->  bit 1 set if refused

serial poll         $00 accepted, $42 refused
```

Caution



This matters more than it may appear. A refused RD leaves the point at its previous coefficient, raises nothing the controller can query, and returns no error to the write — so without the poll it is indistinguishable from success. A procedure that reports the value it sent rather than the value the instrument took will show an operator a table of numbers that were never stored, and the discrepancy will not appear until the next verification.

The mask must be opened with SMS255 **before** EXT CAL is set to ENABLE. SMS is gated to DISABLE, and the mask is applied at the moment a status bit is set rather than when it is read, so a bit masked off during the run cannot be recovered afterwards.

Recommended Validation

The poll says whether the instrument took a value. It says nothing about whether the value was worth taking, and three further checks are needed for that.

- **Plausibility.** Reject a full-scale reading that differs from nominal by more than the acceptance window allows, and a zero reading larger than the window. This is what catches the silent 1000 V failure described in the section "Output Control and Limits" in Chapter 3. There is nothing to be gained by using a wider window than the firmware's own: a reading outside it will be refused, so accepting it locally only defers the failure to a point where it is harder to attribute.
- **Stability.** Require two consecutive readings to agree before entering a value, which catches a disconnected or drifting lead.
- **Completeness of each coefficient.** Measure every point of a coefficient before entering any of it, and enter the group whole or not at all. A range's zero and full-scale points are one such group per polarity, and the three DAC sub-steps are another; see the caution under "Command Sequence" and Chapter 5.

None of these is a substitute for the accuracy specification. They exist to distinguish a measurement from a hardware failure that produces a number. A range that has stopped producing output reads a plausible zero at its zero point, so only a full-scale reading reveals it.

After the Commit

Once EXT CAL is returned to DISABLE the calibration memory can be read again, and the commit should be verified rather than assumed. Three things are worth checking, and they fail independently:

- **That records changed.** Comparing the master block against an image taken before the run shows exactly which coefficients moved. No change at all means the run had no effect — either REND was not accepted, or the entries never reached the working copy. This is a failure and should be reported as one; it is not a remark to be printed alongside a successful verification.
- **That every record is still valid.** A full-scale coefficient of zero or a record that is not valid BCD is the signature of an incomplete or corrupted entry, and neither raises anything on the instrument.

- **That the stored checksum matches the block.** This is the one that costs nothing today and everything later: a mismatch is not detected until the next power-up, at which point the firmware discards the block and substitutes the ROM defaults.

Note



The before-image is worth taking from the master at `$E000` rather than from the working copy at `$DC29`. The working copy is overwritten from the master at boot, so an image of it describes a state that may already have been discarded, and it is the master that the commit writes.

Caution



A valid block is not a changed block, and an accepted entry is not a stored entry. The two checks that feel most reassuring — a count of accepted entries and a correctly checksummed read-back — are both satisfied by an instrument that did nothing at all.

The status byte reports only that the firmware did not *reject* a line. An `RD` that is discarded for want of OPERATE leaves it clean. And the block read back after `REND` decodes perfectly and checksums correctly when it is the untouched original.

Of the three checks above, only the first — that records actually moved — distinguishes a successful adjustment from one that never happened. Verify against the instrument's memory, not against its politeness.

What remains is to leave the instrument as it was found. Standby first, so the terminals are de-energised before anything else is attempted, and then sense and guard back to OFF — the initial values in Table 1-3, and the two parameters a calibration changes that will not reset themselves at the next power-up. The range, the generation value and both output limits need no attention: they are not backed up.

7

Error Codes

GPIB Error Codes

The front panel displays `Err.Gnn` for a GPIB error. Mapping displayed codes back to their originating handler is what identified the transport characteristic in Chapter 2: every error observed during a failed calibration was the correct response to the command that was sent, minus its first character.

The internal code is held at `$DDCB` and can be read over the bus with `MRB`, which is often quicker than catching a code on the panel. The raising sites are a compact block at `$9943` to `$9995`, each loading a code into A and falling into the common store at `$9945`.

Caution



Over codes 4 to 24 the displayed number is the internal code minus three, and it is tempting to treat that as the rule. It is not. The panel label is selected through a jump table at offset `$68` indexed by the code less one, so the mapping is arbitrary and the arithmetic relation stops holding above code 24. The out-of-range code 33 displays as G19, not as G30.

Table 7-1 lists the codes as printed in Operation Manual section A1.5.2. The text of each entry is taken from the tool's own error table, so the two cannot disagree.

Table 7-1. GPIB Error Codes

Code	Meaning
G01	The specified program code does not match the Generator definition.
G02	The string delimiter does not exist.
G03	An invalid channel number (other than 0 to 99) was specified.
G04	The last channel number is less than the first channel number.
G05	An invalid X value (other than 0, 1, 2 or 3) was specified for DLX.
G06	An invalid X value (other than 0 to 255) was specified for SMSX.
G07	An invalid X value (other than 2 to 7 or 1 to 3) was specified for VX or IX.
G08	An invalid X value (other than 10 to 1250) was specified for VLX.
G09	An invalid X value (other than 1 to 125) was specified for ILX.
G10	An invalid X value (other than 1 to 99) was specified for STMX.
G11	An invalid X value (other than 0, 1 or 2) was specified for STX.
G12	No data exists after the D code.
G13	An invalid data was specified after the D code.
G15	A remote control code was received during Ext. CAL.
G16	A VL/IL limit was set in the Divider mode.
G17	Data exceeding 400 characters was entered.
G18	An invalid format of data set was stored in memory.
G20	An invalid remote control code was received in Program mode.
GAd	An attempt was made to set 31 or more addresses in the GPIB Address Setup mode.

Note



A GPIB error also sets bit 1 of the status byte, and that bit is visible by default. Note that `*STB?` is itself subject to the transport characteristic: truncated to `STB?` it raises Err.G11 and therefore sets the very bit it is being used to read.

Codes Absent from the Operation Manual

Section A1.5.2 lists G01 to G13, G15 to G18 and G20. Two codes the firmware raises are missing from it, and both are ones an operator driving a calibration over the bus will meet. Each was confirmed on the front panel of a working instrument and its internal code read back from `$DDCB`. Table 7-2 gives them.

Table 7-2. GPIB Error Codes Absent from the Operation Manual

Code	Internal	Meaning
G14	17	A calibration command was received while EXT CAL was disabled. Raised at $9981, reached by the JP z branch of the gate call described in Chapter 3. It is the exact counterpart of G15, which reports an ordinary command received while EXT CAL was enabled.
G19	33	An out-of-range value was entered with RD. Raised at $4440, which selects this code when the entry arrived over the bus.

Note



G14 is the error a program meets when it begins a calibration without the rear-panel switch having been set. Because the calibration commands are write-only, nothing else reports the condition, and the run appears to proceed while every command is refused.

The Address Error

The Operation Manual lists one further code at the foot of A1.5.2 that does not fit the `Err.Gnn` pattern: `Err.GAd`, raised by an attempt to set 31 or more addresses in the GPIB Address Setup mode. It is internal code 20.

The firmware bears this out exactly. The periodic panel and output update entered at `$2C1F` assembles a byte from the low nibbles of `$D68E` and `$D68F` and, at `$2C5F`, refuses it if the packed BCD value reaches `$31` — the literal 31 of the manual's wording. The value is the one written to the output hardware register at `$F604`.

Note



The label follows from the glyph slots every error display uses: prefix 16, then 10 and 13. Index 16 is G, 10 is A and 13 is d, giving `Err.GAd`. The fan message of a later section fills the same three slots with 15, 10 and 21, which on the same table reads `Err.FAn`. Those two readings are what fix the non-numeric part of the character set; the rest of it was not decoded.

The Complete Code Map

The dispatcher at `$501D` computes `(code - 1) x 2`, indexes a jump table of sixteen-bit addresses at ROM `$0068`, and jumps to a short stub. Each stub loads the label's two digits as `tens x 256 + units` and selects one of four group renderers; the group supplies the prefix letter. Decoding that table gives the whole map, which is reproduced in Table 7-3.

The numbering is not monotonic in the code, which is why an arithmetic rule cannot work: G16, G17 and G18 sit at codes 30, 29 and 31, and G20 at code 19. The table nevertheless yields G01 to G20 exactly once each, with no gaps and no duplicates, and places the five P codes and the four b codes in their own groups — matching the counts printed in the Operation Manual.

Table 7-3. Internal Code to Panel Label

Group	Internal Code	Displayed	Confirmed
GPIB group			
	4	G01	yes
	5	G02	
	6	G03	yes
	7	G04	yes
	8	G05	yes
	9	G06	yes
	10	G07	yes
	11	G08	yes
	12	G09	yes
	13	G10	yes
	14	G11	yes
	15	G12	yes
	16	G13	yes
	17	G14	yes
	18	G15	
	30	G16	yes
	29	G17	yes
	31	G18	
	33	G19	yes
	19	G20	
	20	GAd	
Panel / program group			
	1	P01	
	2	P02	
	3	P03	
	32	P04	
	34	P05	
BCD group			
	25	b01	
	26	b02	
	27	b03	
	28	b04	
Not error labels	21, 22, 23, 24	power-up and status messages	

The Confirmed column marks the 16 codes provoked on a working instrument, with the internal code read back from $DDCB immediately afterwards. Twelve were raised by sending a command with an out-of-range parameter, one by a block longer than 400 characters, one by setting a limit in Divider mode, one by a calibration command with the switch disabled, and one by an out-of-range RD. G14 and G19 were additionally read off the front panel.

Note



Two of those confirmations carry most of the weight. A block over 400 characters returned code 29 and a limit set in Divider mode returned code 30, which the Operation Manual calls G17 and G16 respectively. Those are the two entries where the numbering runs backwards against the code, so an arithmetic rule predicts G26 and G27 and gets both wrong. The jump table predicts them correctly.

Two codes remain unconfirmed. Code 5, G02, is never raised by the firmware at all. Code 18, G15, needs a command sent with the EXT CAL switch enabled, which is a front-panel action. Code 31, G18, has since been confirmed by a malformed memory store; see the MEM entry in Chapter 9.

The Program-Mode Gate

Code 19, G20, is raised at $9989, which loads the code and jumps to the common raiser at $9945. Its condition is a shared gate at $93B5, the counterpart of the EXT CAL gate at $93B0 described in Chapter 3: it returns quietly if $DDF3 is clear and raises G20 if it is set. $DDF3 marks a program run in progress.

The gate is called from the handlers for SC, STM, SEN0, SEN1, GRD0 and GRD1, and the same test is written out in full inside RCL at $8C93 and five other handlers. Those are the commands the instrument refuses while a program is running.

The gate closes only when a program is actually running, and $DDEF — the pause flag, toggled by PAU at $31E8 — is clear. A paused program lets the commands through.

There is a further condition that is easy to miss. In the SEN and GRD handlers the gate is called only when $C0D2 is non-zero, and $C0D2 is the scan mode — which is zero for step scan, ST2. In step-scan mode those commands are therefore never refused. ST is the exception: it raises G20 at $801F with no such test.

Program mode **is** enterable over the bus, contrary to what an earlier edition of this document said. STT reaches the scan engine at $314F, which requires $DDED to be set; RCL is what sets it. The sequence RCL00 then STT was run on the instrument and left it no longer answering MRB, which a single device clear then restored — device clear reaches $9330, which zeroes $DDEB, $DDEF, $DDED and $DDF3 together.

The code itself was still not provoked, in five attempts using both SEN1 and ST0 against a distinctive error baseline. The obstacle is that $DDF3 cannot be observed while a program runs, because MRB does not answer then, so there is no way to confirm from the bus that the gate was ever armed at the moment the command arrived.

Codes 20 to 24 reach a renderer with digit pairs that are not decimal, so they are not Err.xnn labels. Three are raised from the power-up self-test and report the conditions that *TST? also reports. Table 7-4 gives what each one checks and what the instrument does about it.

Table 7-4. Self-Diagnostic Messages

Code	Raised At	Condition and Action
21	$0220	ROM sum check error. The firmware sums the program ROM against the byte at $BFF0. Manual A1.5.4 says the display continues, and the firmware agrees: it enters an endless loop at $0226 and the instrument does not proceed.
22	—	RAM read/write error; the display continues. No instruction in firmware A05 stores this code, and there is no read/write test in the boot path; the 55/AA test appears only in the *TST? handler at $8853. The message is documented but not reachable by the $DDCB route.
23	$0278	Battery-backed RAM data destroyed. An eight-byte doubling pattern at $DE7E is verified. Displayed about five seconds, the RAM parameters are initialised and operation continues.
24	$02A4	Calibration data in the EEPROM destroyed. The 364 bytes at $E000 are summed against $E16C. Displayed about five seconds, the ROM defaults are substituted and operation continues, out of adjustment.

These four are the conditions `*TST?` also reports, and the correspondence is one to one: bit 0 the ROM sum, bit 1 the RAM read and write, bit 2 the RAM parameters, bit 3 the EEPROM data. Bit 4, the fan, is the subject of a later section.

Note



Two entries in Table 7-3 were confirmed directly: internal code 17 displayed G14 and code 33 displayed G19, each read back from `$DDCB` with `MRB` immediately after the error was seen on the panel. Code 32 mapping to P04 is what makes the pairing in the previous section firmware-certain rather than inferred: the two branches at `$4440` select codes 32 and 33, which this table shows are P04 and G19.

Exactly two of the thirty-four codes are never raised. Every instruction in the ROM that can reach `$DDCB` was enumerated — twenty-one direct stores and the common raiser at `$9945`, with the constants on both arms of every branch that feeds them — and the reachable set is every code from 1 to 34 except 5 and 22. Code 5 is G02, "the string delimiter does not exist", which the Operation Manual documents; code 22 sits among the power-up messages. Neither condition is reachable in this revision.

The Status Byte

Errors are also reported through the status byte, which manual 4.7.1 defines as shown in Table 7-5. Bits 3, 5 and 7 are always zero.

Table 7-5. Status Byte, R6161 Mode

Bit	Meaning
0	Limit operation. Set when an output limit occurs, cleared when it is released.
1	Command syntax error. Set by an undefined program code, a parameter outside its limit, or a code that is too long; cleared when a correct code is received.
2	Program end. Set when the last channel of a program-mode run completes.
4	Fan stop error.
6	RQS.

The mask set by `SMS` governs which of these reach the byte, and it powers up at 255 with every bit enabled. Errors are therefore visible to a polling program by default, and it is `SMS0` that hides them. Measured on the instrument: with the mask at 255 an invalid command left `*STB?` reading 66, which is bit 1 with RQS; with the mask at 0 the same command left it reading 0. `*CLS` clears the byte and does not disturb the mask.

Two Different Masks

The serial-poll value and the *STB? response are not the same quantity. The routine at $5F0A forms the poll value as the raw status at $DDD7 ANDed with the SMS mask and then with $17, setting RQS if anything survives, and leaves it at $DDDF. The *STB? handler at $892E then returns the raw status, with bit 6 forced, ANDed with that value and with $47.

```
serial poll :  status & mask & $17      bits 0, 1, 2, 4

*STB?       : (status | b6) & $DDDF & $47   bits 0, 1, 2, 6
```

Caution



The fan bit is reachable only by serial poll. $17 includes bit 4 but $47 does not, so a fan stop raises the bit in the poll response and is invisible to *STB?. A program that monitors instrument health by polling *STB? will never see it, and the only other indications are the panel and the substituted reply described in the next section.

This is a documentation error, not merely an omission. Operation Manual Figure 4-5 draws the status byte as a single register read equally by the serial poll and by *STB?, with bit 4 present in both. The firmware does not implement it that way: the two paths apply different masks, and only the poll path passes bit 4. Any procedure written from the figure — including a health check that expects a fan stop to appear in the *STB? response — will silently fail to detect the condition.

Note



The two expressions reproduce the measurements exactly. With the mask at 255 a syntax error gives (0x02 | 0x40) & 0x42 & 0x47 = 66, the value observed; with the mask at 0 the poll value is zero and so is the response.

Bit 1 and the Error Path

Bit 1 is not merely a syntax indicator. It is set by the same routine that raises every Err.G code, which makes it the one channel through which a refused command can be detected programmatically.

The internal error code is written to $DDCB at 21 places in the image, and every one of them calls $4FCD on the next instruction; there is no path that sets a code without it. $4FCD reaches the compare chain at $4FF7, which sets bit 1 through $5E46 for internal codes $04 to $14 and for $1D, $1E, $1F and $21. That is exactly the Err.Gnn family. The boot self-test codes and $20, the front-panel twin of G19, raise no bit.

The bit is latched to the outcome of the **most recently executed command line** and is cleared only when the next line completes without error. The GPIB input handler does this at $7715: it executes the whole line, then tests the "this line errored" flag at $D60C and calls $5ED9 to clear bit 1 if the flag is clear.

```
write "RV4+FS"   clean line          ->  bit 1 cleared

write "RD1010.1"  refused with G19    ->  bit 1 set, RQS set

serial poll                            ->  $42

serial poll again, no command between  ->  $42   (not self-clearing)

write "RV4-0"     clean line          ->  bit 1 cleared
```

A poll before the command under test is therefore unnecessary, and a poll after it is unambiguous. The serial-poll handler at $75CC explicitly declines to drop RQS while bit 1 is set, and never clears the bit itself, so reading the byte does not destroy the evidence.

Note



Verified on the instrument on 28 July 2026 with the mask at 255. A baseline poll read `$00`; `H` left it at `$00`; an undefined command (G01, internal code 4) took it to `$42`; `H` again returned it to `$00`; `D+1000` on the 1 V range (G13, internal code `$10`) took it to `$42`; two further polls with no command between still read `$42`; and a valid `D` cleared it.

Caution



Nothing on this path is gated by EXT CAL. `$4FCD`, `$5E46`, `$5F09`, `$5F1A` and `$75CC` contain no call to the gate at `$93B0`, and the poll itself is answered by the GPIB controller from register `$F605` rather than by the command dispatcher. The serial poll is consequently the **only** thing the instrument will still tell a controller while a calibration is in progress. Chapter 6 depends on this.

The mask must be opened before the switch is moved. `$5F09` applies the mask when a bit is *set*, not when it is read, so a bit masked off at the moment of the error is lost permanently — and `SMS` is itself gated to EXT CAL = DISABLE.

The Fan Message

One fault is reported as text over the bus rather than as a code on the panel. The string `Error.FAN !!` occupies twelve bytes at `$9DFB` and is copied into the response buffer by the routine at `$964C`. It is the only ASCII message string anywhere in the firmware; every panel indication is assembled from glyph indices instead, which is why nothing else of the kind exists.

`$964C` has exactly two callers: `$85D1` in the `D?` handler and `$8729` in the `PANE?` handler. Both branch to it when the flag at `$C00B` is set, and both then return the string in place of their normal reply.

Caution



A program that reads `PANE?` must be prepared for a reply that is not a parameter record. The substitution is silent: no error code is set, the status byte is unaffected, and a parser expecting comma-separated fields will fail in a way that resembles a communication fault rather than an instrument fault.

The detection is a hardware input. `$0438` samples `$FEC0` and, at `$044D`, jumps to `$22ED` when bit 4 is high. That routine stashes the current display, loads an eight-byte panel message from `$9AA7` and latches `$C00B`. The last four bytes of that message are byte-identical to the framing every `Err.xnn` display uses, and the remaining three fill the prefix and two digit slots with indices 15, 10 and 21, which read `Err.FAn`.

Manual A1.5.4 lists the condition among the self-diagnostic messages as "the fan has stopped", and gives one instruction with it.

Warning



The Operation Manual's stated action for a fan stop is to turn the POWER switch off immediately. Unlike the other self-test failures, which either halt or substitute defaults and carry on, this one calls for the instrument to be shut down. It is not a fault to investigate with the instrument running, and deliberately stalling the fan to observe the behaviour is not advisable.

That `$C00B` is the fan flag is not an inference: the `*TST?` handler reads it at `$8879` and sets bit 4 of the self-test byte from it, and manual Table 4-5 item (9) documents bit 4 as the fan stop error.

Note



Recovery restarts the instrument. When the condition clears, the path at `$20EE` clears `$C00B` and executes `RST 0`, re-entering the firmware from address zero. A fan fault is therefore not something a running program can ride out: the instrument reboots when the fan returns, and the working calibration copy is reloaded from the master as at any power-up.

Panel and Program Error Codes

The front panel displays `Err.Pnn` for an error in data entered from the panel or held in a program channel. These errors are raised by the panel and program-mode entry paths rather than by the GPIB parser, so they are not reported through the status byte. Table 7-6 lists them as printed in Operation Manual section A1.5.

Table 7-6. Panel and Program Error Codes

Code	Meaning
P01	A generation data setup error.
P02	A limit data setup error.
P03	First or last channel error in Program mode. The first channel must not exceed the last.
P04	An out-of-range error of EXT CAL data.
P05	A channel exceeding two digits was specified in Recall mode.

`Err.P04` and `Err.G19` are the same condition. The handler at `$4440` tests the local/remote flag at `$C0D3` and stores internal code 32 when the entry came from the front panel or 33 when it came over the bus:

```
LD   A,($C0D3)
OR   A
LD   A,$21          ; over the bus   -> displays G19
JR   nz,$444A
LD   A,$20          ; from the panel -> displays P04
$444A:
LD   ($DDCB),A
```

Note



This is why the Operation Manual documents `P04` and has no G-group equivalent: its calibration chapter contemplates the procedure being carried out from the front panel only. A program driving the same procedure over the bus will see `G19` instead, for the same rejected value.

Both halves were observed. The same out-of-range value was entered twice on one instrument: over GPIB it displayed `Err.G19`, and keyed in from the front panel it displayed `Err.P04`.

BCD Remote Control Error Codes

The front panel displays `Err.bnn` for an error raised by the optional BCD remote-control interface described in Chapter 5 of the Operation Manual. The first three are mode conflicts: combinations of range, divider and function that the hardware cannot present at once. Table 7-7 lists them.

Table 7-7. BCD Remote Control Error Codes

Code	Meaning
b01	The 1000 V range was selected and the divider was turned on.
b02	The 1000 V range was selected and the I mode was selected.
b03	The divider was turned on and the I mode was selected.
b04	An illegal data was set.

The same three conflicts exist over GPIB, where they are reported as Err.G16 by the range and limit handlers rather than as a b-group code. That is the error a program meets when it attempts to set a voltage or current limit while a divider range is selected, as described in the section "Output Control and Limits" in Chapter 3.

8

Verification Record

The statements in Chapters 3 to 6 were confirmed by performing a complete adjustment over GPIB and measuring the result against a Keysight 3458A.

Coefficient Decode

With all coefficients deliberately set to nominal, so that no correction was applied, the output was predicted from the previously stored coefficients and compared with measurement. The prediction combines a full-scale record and a zero record — two different formats at two different addresses — across a 1000 : 1 span. Table 8-1 gives the result.

Table 8-1. Predicted Against Measured Output

Point	Predicted	Measured	Difference
1 V range, commanded 1 V	1.002379 V	1.0023907 V	12 ppm
1000 V range, commanded 1000 V	1001.976 V	1001.98817 V	12 ppm

Adjustment and Re-verification

A full 45-point adjustment was then performed. Each measured full-scale value minus its measured zero reproduced the coefficient that had been stored in the instrument beforehand, over all twenty full-scale points, to within 17 ppm and mostly within 10 ppm — an independent confirmation both of the decode and of the instrument's stability.

The adjustment was committed and the instrument re-verified. Table 8-2 gives the result.

Table 8-2. After Adjustment

Range	Point	Commanded	Measured	ppm of F.S
1 V	+F.S	1 V	1.0000006 V	+0.60
1 V	Zero	0 V	1.8377606e-06 V	+1.84
1 V	-F.S	-1 V	-1.00000039 V	-0.39
10 V	+F.S	10 V	9.99998586 V	-1.41
10 V	Zero	0 V	8.71260617e-06 V	+0.87
10 V	-F.S	-10 V	-9.99999839 V	+0.16
100 V	+F.S	100 V	99.9999137 V	-0.86
100 V	Zero	0 V	0.000105423352 V	+1.05
100 V	-F.S	-100 V	-99.9999722 V	+0.28
1000 V	+F.S	1000 V	999.995956 V	-4.04
1000 V	Zero	0 V	0.00299539865 V	+3.00
1000 V	-F.S	-1000 V	-999.996858 V	+3.14

The worst deviation is 4.04 ppm of full scale, against +1870 to +2063 ppm before the adjustment. The supporting points also agreed with the manual: the voltage limit measured 499.79 V against a stated approximately 500 V, and the current limit 50.383 mA against the manual's worked example of 50.75 mA.

Confirmed by Observation

- `REND` performs a real EEPROM commit: the master and the working copy both changed, identically.
- **The voltage limit does not gate the calibration path.** With the limit set to its minimum of 10 V, the 1000 V full-scale point still produced 1001.99 V — the expected uncorrected full scale. The procedure drives the output from the selected point rather than through `D`, and the limit does not apply to it.
- **The third array holds one DAC coefficient and the two limit coefficients,** not three DAC coefficients. Each was entered on its own and only the expected record moved.
- **The coefficient relation is** `100 x (B - A) / (D - B)`, confirmed against four deliberately chosen entry sets. The coefficient rounds; both limit records truncate.
- `RD` **requires the point to be in OPERATE.** The selector alone is not enough, and an entry made without it is discarded with no error of any kind. No settling delay beyond the controller's own command overhead, about a second, is needed for acceptance.
- **Selecting a point takes it out of OPERATE.** Eight entries were made under otherwise identical conditions, four preceded by `E` and four by a bare re-selection. All eight were accepted on the status byte; only the four made in OPERATE changed the working copy. A complete 45-point run built on the re-selection alone was accepted in full, stored nothing, and committed an unchanged block that read back valid and correctly checksummed.
- **An** `RD` **still stores after the controller has talked to another instrument.** A full integration cycle on a 3458A between the measurement and the entry left the R6161's selected point and OPERATE state intact, so a point may be entered immediately after it is measured.
- **The three DAC sub-steps are one stored record.** `RKTA`, `RKTB` and `RKTD` fold into the single coefficient at `$DD7D`: entering two of the three leaves the working copy unchanged, which is indistinguishable from a failed entry unless the whole chain is completed.

- `RD` **accepts a value within 1 % of the range's full scale of the point's nominal** ***at a range point,*** on both sides, and rejects anything outside. Confirmed to the seventh digit on a full-scale point, a zero point, a divider point and a current point; Table 9-2 gives the edges. The +/-0.100000 V edge recorded at a DAC point belongs to `RKTA` alone and is a different test; see Table 6-3.
- **G14 and G19 exist** and are absent from the Operation Manual. Both were displayed on the panel and their internal codes read back from `$DDCB`.
- **Err.P04 and Err.G19 are the same rejection.** The identical out-of-range value produced G19 when entered over the bus and P04 when entered from the front panel, matching the two branches at `$4440`.
- **The complete internal-code map** was recovered from the jump table at `$0068` and yields G01 to G20, P01 to P05 and b01 to b04 with no gaps or duplicates; see Table 7-3.
- **Internal code 20 is** `Err.GAd`**,** the address error at the foot of manual A1.5.2. The bound in the firmware is a comparison against `$31`, matching the manual's "31 or more addresses" exactly.
- **Codes 21 to 24 are the self-diagnostic messages of manual A1.5.4** and correspond one to one with `*TST?` bits 0 to 3. The action the firmware takes at each matches the action the manual states, including the ROM sum error halting the instrument.
- **Entering a full-scale point without its zero** leaves that range's coefficient at zero, silently, and a subsequent `REND` commits it: the commit at `$21A2` copies the whole working block and `$402E` recomputes the checksum over it, so neither the power-up check nor `*TST?` detects the damage.
- **The master block carries a checksum** at `$E16C`, the low eight bits of the sum of its 364 bytes. On the instrument examined the block summed to $96 and the stored byte read $96.
- **Codes 21, 23 and 24** are the power-up ROM, battery-backed data and calibration checksum failures; a ROM failure halts the instrument. See Table 7-4.
- `*IDN?` returns ADVANTEST,R6161,REV A05, against the REV A01 given as the example response in manual Table 4-5.
- `D?` **returns the output setting** in the same form as the value field of `PANE?`, reported in STANDBY as well as in OPERATE. It is a parameter read, not a measurement.
- **The status-byte mask powers up at 255,** so errors are visible to a polling program by default; `SMS0` is what hides them. An invalid command gave `*STB?` = 66 with the mask at 255 and 0 with the mask at 0.
- `RCL` **leaves the transport faulted more often than other writes.** One interposed read left the following command answered in 17 of 20 attempts, two reads in 29 of 30, and extra delay made little difference.
- **An unknown command raises internal code 4**, read back from `$DDCB` and displayed as G01, a third independent check of the code map.
- `MRB` reads processor memory over the bus, and `MWB` is accepted; neither appears in the Operation Manual.
- `MEM` **clamps the voltage limit to 130** on every range but the 1000 V range. Measured on the 1 V range: `VL0100` stored as 100, `VL0130` as 130 and `VL0140` as 130. An earlier edition of this document reported the field as never stored, which was wrong — every test had used a value above the clamp.
- **The two limit fields of** `MEM` **are not independent.** `IL` without `VL` raises Err.G18 — the seventeenth G code confirmed on hardware — and `VL` without `IL` raises Err.G01.
- `RCL` **enters program mode** rather than recalling a setup. `$DDED` read 0 before `RCL00` and 1 after, and `D` with a value returned it to 0.

- **Program mode can be entered over the bus** with `RCL` followed by `STT`. The instrument then stops answering `MRB`, and a single device clear stops the program and restores it.
- **The fan bit is unreachable by `*STB?`,** contradicting manual Figure 4-5. The serial poll masks with `$17` and `*STB?` with `$47`; only the first includes bit 4.
- **The complete 45-point routine runs end to end over the bus.** Rehearsed on 29 July 2026 against a working instrument with every entry confirmed by serial poll: 45 of 45 accepted, no skips, no refusals and no half-entered units, across all four hookup groups including the 1000 V range and the 500 V limit step. `REND` was blocked in software and the EEPROM master read back byte for byte identical afterwards, confirming that entries reach the working copy alone.
- **A refused `RD` is visible to a serial poll while EXT CAL is ENABLE.** Measured 29 July 2026 with the point energised: `RD+5.000000` at the 1 V zero point gave poll $00, and `RD+20.00000` -- twice the band -- gave poll $42. This is the channel an automated calibration depends on, and it is the only one: every query is refused in that position.
- **The range acceptance band is exact to the last digit.** On the 1 V range, full scale accepted 1010.000 and refused 1010.001, accepted 990.0000 and refused 989.9999, and refused a negative value at a positive point; the zero point accepted 10.00000, refused 10.00001, and accepted -10.00000, confirming that zero points do not check the sign.
- **The DAC chain rules hold.** RKTA refused 0.15 V against its 0.1 V ceiling and accepted -0.01 V; RKTB refused a value below RKTA's, refused a negative one, and refused 0.25 V against the 0.2 V ceiling; RKTD refused a value not exceeding RKTB's.
- **Both limit windows are confirmed at both edges.** RVL accepted 400.0000 and 500.0000 and refused 380.0000 and 620.0000; RIL accepted 50.00000 and 60.00000 and refused 38.00000 and 62.00000.
- **`RD` entries reach the working copy but not the master.** After forty entries with EXT CAL ENABLE and no `REND`, the EEPROM master read back byte for byte identical and its checksum was unchanged at $96, while the working copy had moved.
- **Status-byte bit 1 is set by every `Err.G` code and is latched to the last command line.** A clean line read `$00`; an undefined command (G01) and an out-of-range `D` (G13) each took it to `$42`; the next clean line cleared it; and repeated polls with no command between did not clear it. This is the channel that reports a refused `RD` during a calibration, when every query is gated off.
- **The 1 V range stores its full-scale coefficient in volts** although it is entered in millivolts. Read from the EEPROM master of an instrument in adjustment, alongside nine other ranges that all store in their entry unit and all sit within 0.14 % of nominal; Table 5-4.

Note



Every entry test above was made without sending `REND`. Because the working copy is reloaded from the EEPROM master at every power-up, a power cycle restored the instrument exactly: after each session the full 364-byte working copy was read back and compared byte for byte against the image committed by the last real adjustment, and matched.

Not Verified

The correspondence between the three RKT selectors and the three sub-steps named in manual 7.2.2 rests on the DAC drive patterns at `$42DF`, `$42EF` and `$42FA`, which establish that A is a base point, B differs from it by one coarse-register step and D from B by one fine-register step. That the manual's own wording for each sub-step matches those roles is a reading of the manual, not an observation of the instrument, and the panel indication shown at each selector has not been recorded.

One behaviour was observed once and has not been reproduced. During a session in which no RKT point was entered, the stored DAC coefficient changed from 9979 to 9978. Seven attempts to reproduce it have failed: an out-of-range rejection on its own; an accepted `RVL` on its own; RKT entries followed by an accepted `RIL`; RKT entries followed by a rejection and then an accepted `RVL`; both limits entered with no RKT entries at all; the original sequence itself, three consecutive rejections followed by both limits accepted; and that same sequence again with the twenty-second dwells of the original restored. In every case the coefficient held where it had been left.

The arithmetic narrows what the value can have been. Working through the conversion in Chapter 5, the stored figure is *floor*((quotient × 100 + 50) / 100), so 9979 requires a quotient of 9978.90 and 9978 requires one no greater than 9978.49. Those differ by more than forty parts per million, far too much to be a rounding wobble on the same inputs. 9978 is not a truncated form of the entries that were in the buffers; it is exactly what the values from the instrument's own last real adjustment produce, and that is also the figure held in the EEPROM master.

What the Firmware Allows

A complete search of the ROM bounds the problem tightly. The address `$DD7D` appears in exactly three instructions: a read at `$0A70`, where the coefficient is fetched to compute the output; the eight-byte commit to the master at `$0C90`; and the store at `$4383` that ends the computation. Nothing else addresses it. The three block copies are equally well bounded — `$218F` master to working, called only from `$0257` and `$0294`; `$5810` ROM defaults to working, called only from `$025C` and `$02AA`; `$21A2` working to master, called only from `$1ED7`, the `REND` handler. The first four are boot code. **There is no runtime path by which the working copy can be reloaded, and the computation is the only thing that can write the coefficient.**

That disposes of the reload hypothesis, but it also removes the difficulty that made the observation look impossible. The premise was that no `RKT` point had been entered, so nothing should have run the computation at all. That premise is wrong. As Chapter 5 now records, `$DE45` is cleared by two front-panel key handlers, both taken only while EXT CAL is enabled, and `$DE28` remains set until the sequence completes. The computation can therefore run repeatedly during a session, on key events that have nothing to do with entering a point. A recomputation in that window is not only possible, it is expected.

What the firmware does not allow is the value that appeared. The arithmetic is exact integer arithmetic on the three entry buffers, with no shared state that anything else can reach: `$C0A5` is written at one instruction and read at one other, and the three stages are called as a unit from `$0C5E` with no branch into the middle of the sequence anywhere in the ROM. The buffers at the time held 10000, 109789 and 110789, which give 997890000 / 1000 = 997890 and a stored 9979. To reach 9978 the quotient must fall between 997750 and 997849, and no divisor consistent with those buffers does that. The operands must have been different. The values that produce 997846 exactly are 12733, 112817 and 113820 — the instrument's own previous adjustment.

Two further checks were made against the session record. There was no power cycle in the window: the same VISA session and the same process spanned it, no `*IDN?` was re-issued, and the first power cycle of the day came fifty-two minutes later. And the two limit records read master values in the same window not because anything reverted but because the values entered, 499.7961 and 50.38791, truncate to exactly the figures already stored. They carry no information, which leaves the coefficient as the only anomalous reading.

So the account is now specific rather than open. A recomputation ran, as the firmware permits; it produced a result that requires the previous adjustment's operands; and the buffers read back four minutes later held the session's own. One of those three statements is false, and the evidence does not say which.

Why the Reproduction Attempts Could Not Have Worked

One thing can now be said about the seven failed attempts: they were incapable of reproducing the effect, whatever its cause. Every one of them was driven entirely over GPIB, and as Chapter 5 records, the only route from the bus to clearing `$DE45` is a point selector, which clears `$DE28` with it and so closes the gate instead of re-arming it. A spurious recomputation cannot be provoked from a program. The mechanism that produces one without a point entry exists only in the two front-panel key handlers.

The operator was at the panel throughout the window in question, reading the display and working the EXT CAL switch. That makes an unrecorded key press the leading explanation for the recomputation itself, and it redefines the reproduction test: replaying the bus traffic will never show anything, whereas replaying it together with the panel actions might.

Note



This does not account for the value. A recomputation from the buffers that were present gives 9979 whether it is triggered from the panel or not. What has changed is that the missing step is now known to lie outside the bus traffic, which is the only part of the session that was recorded.

The effect is one count in about ten thousand, so it does not matter to a calibration, but it means the coefficient written by a session cannot be assumed to depend only on the three RKT entries. A reader relying on the relation in Chapter 5 should read the record back after the run rather than compute it.

Note



The test that would settle it is now a narrow one. Read the three entry buffers and `$C0A5` immediately before and immediately after each `RD`, together with `$DE28`, `$DE45`, `$DE34` and `$DE35`. A recomputation is visible as `$DE45` going from 0 to 1, and the operands that produced it are on the record. Sampling only the coefficient, as the original session did, cannot distinguish the three statements above.

Two codes are defined in the map of Table 7-3 but are never raised: G02 at internal code 5, and code 22 among the power-up messages. That is now an enumeration rather than an impression — every instruction that can reach `$DDCB` was listed and the reachable set is 1 to 34 less those two — but neither condition was provoked, so their labels rest on the jump table alone.

Internal code 20 is described in Table 7-4 from its firmware path, but the instrument state that would provoke it was not identified and the code was never seen. The description there rests on reading the check, not on observing it fire.

9

Command Reference

Introduction

This chapter describes every mnemonic the R6161 firmware dispatches, as a set of command entries. It covers the documented listener and query codes of Operation Manual sections 4.5 and 4.6, the calibration commands of Chapter 3 of this document, and the two memory-access commands that appear nowhere in the Operation Manual. Entries use the same form as the manual's own command descriptions.

Each entry begins with the mnemonic, followed by one line for each syntax form the firmware accepts, then a description. The last line of each entry records the handler address, the EXT CAL switch position the handler requires, and the section of the Operation Manual that describes the command, or *not documented* where none does. Handler addresses are for firmware revision A05.

Notation

The following conventions are used in the syntax lines of this chapter.

`< >`	Angular brackets enclose a parameter that the programmer supplies. The text of the entry states its permitted values.
`{ | }`	Braces enclose a set of alternatives, of which one and only one may be given. The vertical bar reads as "or".
`[ ]`	Square brackets enclose an optional element.
…	An ellipsis marks an element that may be repeated.

No space is permitted between a mnemonic and its parameter. A comma separates commands within a block and may be omitted where the result is unambiguous, as described in manual 4.5.1(2). `(No query)` marks a command that has no query form; `(Query only)` marks a mnemonic that exists only as a query.

Note



Every command in this chapter is subject to the transport characteristic described in Chapter 2. A serial poll must be interposed after each command, or the next query will go unanswered.

Dispatch Index

Table 9-1 was extracted programmatically from the R6161-mode and TR6120A-mode comparison chains in firmware A05. Dispatch is the address of the comparison that recognises the mnemonic; Handler is the routine it jumps to. The EXT CAL column is derived from the presence and sense of the gate call described in Chapter 3; *either* means the handler contains no gate and works in both switch positions.

Where a mnemonic is a prefix of a longer command — `RV` and `RVL`, or `V` and `V4` — the comparator matches the prefix and the handler parses the remainder itself. Such commands have no dispatch entry of their own, and the gate may be applied in the per-argument branch rather than at the handler entry. Those cases are marked in the notes and are described in full in the entries that follow.

Table 9-1. Complete Command Dispatch Table

Mnemonic	Dispatch	Handler	EXT CAL	Mode	Notes
*CLS	$7A31	$7BC1	DISABLE	R6161	
*IDN?	$7A3A	$86C2	DISABLE	R6161	
*RST	$7A43	$7FAE	DISABLE	R6161	initialises battery-backed data - do not send
*STB?	$7A55	$8920	DISABLE	R6161	
*TRG	$7A5E	$7C97	DISABLE	R6161	shares its handler with STT
*TST?	$7A67	$880F	DISABLE	R6161	
C	$792C	$7BA1	DISABLE	R6161	
D	$7A80	$8D95	DISABLE	R6161	
D?	$7935	$859D	DISABLE	R6161	
DL	$7A77	$7BD9	DISABLE	R6161	
E	$793E	$7F5D	either	R6161	
GRD0	$7950	$7E09	either	R6161	
GRD1	$7959	$7E33	either	R6161	
GRD?	$7962	$869F	DISABLE	R6161	
H	$796B	$821F	either	R6161	
I	$7A92	$8380	DISABLE	R6161	gate applied per range digit
IL	$7A89	$7CE0	DISABLE	R6161	
MEM	$7A9B	$8B55	DISABLE	R6161	
MRB	$7AA4	$8BEE	DISABLE	R6161	not documented: read one byte of memory
MWB	$7AAD	$8C30	DISABLE	R6161	not documented: write one byte of memory
OP	$7974	$7F5D	either	R6161	synonym for E
PANE?	$797D	$86DC	DISABLE	R6161	
PAU	$7986	$7CBC	DISABLE	R6161	
RCL	$7AB6	$8C8D	DISABLE	R6161	
RD	$7ABF	$8D19	ENABLE	R6161	requires a preceding point selection
REND	$798F	$7F83	ENABLE	R6161	commits the calibration to EEPROM
RI	$7AC8	$809A	ENABLE	R6161	gate per range digit; all branches require ENABLE
RKTA	$7998	$8034	ENABLE	R6161	not documented: DAC connection point
RKTB	$79A1	$8052	ENABLE	R6161	not documented: DAC connection point
RKTD	$79AA	$806C	ENABLE	R6161	not documented: DAC connection point
RV	$7AD1	$80EC	ENABLE	R6161	gate per range digit; all branches require ENABLE
S0	$79B3	$824D	DISABLE	R6161	shares its handler with SRQON
S1	$79BC	$822F	DISABLE	R6161	shares its handler with SRQOF
SB	$79C5	$821F	either	R6161	synonym for H
SC	$7ADA	$7C1E	DISABLE	R6161	
SC?	$7947	$861F	DISABLE	R6161	

Mnemonic	Dispatch	Handler	EXT CAL	Mode	Notes
SEN0	$79CE	$7E5E	either	R6161	
SEN1	$79D7	$7E93	either	R6161	
SEN?	$79E0	$87EC	DISABLE	R6161	
SMS	$7B07	$82A4	DISABLE	R6161	
SMS?	$7A4C	$88D0	DISABLE	R6161	
SRQ?	$79FB	$88F0	DISABLE	R6161	
SRQOF	$79E9	$822F	DISABLE	R6161	
SRQON	$79F2	$824D	DISABLE	R6161	
ST	$7AEC	$7FEC	DISABLE	R6161	
ST?	$7A04	$87C2	DISABLE	R6161	
STM	$7AE3	$8277	DISABLE	R6161	
STM?	$7A0D	$866E	DISABLE	R6161	
STP	$7A16	$7C80	DISABLE	R6161	
STT	$7A1F	$7C97	DISABLE	R6161	
V	$7AFE	$835B	either	R6161	gate per range digit; all branches require DISABLE
VL	$7AF5	$7D1C	DISABLE	R6161	
Z	$7A28	$7FAE	DISABLE	R6161	initialises battery-backed data - do not send
I1	$7B2E	$84B9	DISABLE	TR6120A	
I2	$7B37	$84E2	DISABLE	TR6120A	
I3	$7B40	$850B	DISABLE	TR6120A	
L	$7B49	$7EC9	either	TR6120A	TR6120A limit selection L1 to L8
V4	$7B64	$83E8	DISABLE	TR6120A	
V5	$7B6D	$8412	DISABLE	TR6120A	
V6	$7B76	$843C	DISABLE	TR6120A	
V7	$7B7F	$8466	DISABLE	TR6120A	

61 distinct dispatch entries are listed. The mnemonics reached through a prefix match — the calibration point selectors and the TR6120A-mode range codes — raise the number of distinct commands above this figure.

Calibration Commands

The commands in this section are accepted only while the rear-panel EXT CAL switch is in the ENABLE position. None of them appears in the Operation Manual. They are listed in the order in which a calibration uses them: a point selector, then RD to enter the measured value, and finally REND to commit.

RV

RV<d>{+|-}{0|FS}

Selects a voltage calibration point and drives the output to it. <d> is the range digit 2, 3, 4, 5, 6, 7 or 9, with the meanings given in Table 1-1. The sign selects the polarity of the point and the suffix selects its magnitude: **0** for the zero point and **FS** for full scale.

The command takes no value of its own. The point is measured on the reference standard and the result is entered with RD. Selecting a point sets the two state variables that RD tests, so an RD that does not follow a selector is discarded.

The output must be energised with E before the point can be measured. Ranges 2, 3 and 9 appear on the divider terminals; the others appear on the main output terminals. (No query)

Caution



A range's zero point and its full-scale point must both be entered. Entering the full-scale point alone leaves that range's coefficient at zero in the working copy, silently.

Dispatch $7AD1 • handler $80EC • EXT CAL = ENABLE • Not documented in the Operation Manual.

RI

RI<d>{+|-}{0|FS}

Selects a current calibration point and drives the output to it. <d> is the range digit 1, 2 or 3, for the 1 mA, 10 mA and 100 mA ranges. The sign and suffix have the same meaning as for RV. The command takes no value; the measurement is entered with RD in milliamperes. (No query)

Dispatch $7AC8 • handler $809A • EXT CAL = ENABLE • Not documented in the Operation Manual.

RVL

RVL

Selects the voltage-limit calibration point. The command takes no range digit, no sign and no value: there is one such point. The instrument drives its voltage limit to approximately 500 V, which is measured on the reference standard and entered with RD in volts.

The mnemonic has no dispatch entry of its own. It is reached by prefix match on RV at $7AD1, and the limit branch within that handler is at $8200. The handler sets a flag at $DE3F to mark the limit point as the selected one.

The entered value is stored at $DD8D in hundredths of a volt, **truncated** rather than rounded: an entered 495.5560 V, which scales to 49555.60, was stored as 49555. (No query)

Handler $8200, reached from $80EC • EXT CAL = ENABLE • Not documented in the Operation Manual.

RIL

`RIL`

Selects the current-limit calibration point, in the same manner as `RVL`. The instrument drives its current limit to approximately 50 mA. Manual 7.2.2 directs that this be measured as the voltage across a 10 Ω standard resistor connected across the output, and that the current calculated from that reading be entered with `RD` in milliamperes.

Nothing in the firmware requires the reading to arrive by that route. The value entered is a current in milliamperes however it was obtained, and the 40 to 60 mA acceptance window at $3FF7 is applied to that number alone. A reference multimeter's own current terminals are a low enough impedance that the instrument is in limit with them in series, so the current through the meter *is* the limit current and can be entered directly — one fewer term than V / R, and no resistance that has to be known. This is not the manual's method and is offered as the fallback for a bench with no 10 Ω standard; it has not been checked against the manual's method on the same instrument.

The mnemonic is reached by prefix match on `RI` at $7AC8; the limit branch is at $81E1 and sets a flag at $DE40.

The entered value is stored at `$DD85` in hundredths of a milliampere, **truncated** rather than rounded: an entered 48.55560 mA, which scales to 4855.56, was stored as 4855. `(No query)`

Handler $81E1, reached from $809A • EXT CAL = ENABLE • Not documented in the Operation Manual.

RKTA / RKTB / RKTD

`RKTA`

`RKTB`

`RKTD`

Select the three DAC connection calibration points. All three are measured on the main output terminals with the instrument on the 10 V range, and each is entered with `RD` in volts.

The three are not three independent coefficients. They are three measurements of the same output at three DAC settings, and the firmware reduces them to the single value stored at `$DD7D`. The drive patterns give each point its role: `RKTA` sets the coarse and fine registers to `$1050` and `$1050`, `RKTB` advances the coarse register to `$1150`, and `RKTD` then advances the fine register to `$7050`. A is the base point, B one coarse step above it, D one fine step above B.

The stored coefficient is `100 x (B - A) / (D - B)`, scaled by 10^6 and rounded. Chapter 5 gives the arithmetic and the confirming measurements.

These three points are taken first in a full adjustment, because every range coefficient is subsequently measured through the DAC that they describe. `(No query)`

Dispatch $7998 / $79A1 / $79AA • handlers $8034 / $8052 / $806C • EXT CAL = ENABLE • Not documented in the Operation Manual.

RD

`RD{+|-}<numeric_value>`

Enters the measured value of the currently selected calibration point. <numeric_value> carries up to seven digits and may include a decimal point. A leading `+` may be given or omitted; `-` is required for a negative value.

The value is given in the entry unit of the selected point, which is the panel unit of its range as listed in Table 1-1 — millivolts on the divider ranges, volts on the 1 V to 1000 V ranges, milliamperes on the current ranges. This is not the unit used by the `D` command, which takes volts on every voltage range.

The point must be in OPERATE. The handler tests two state variables before accepting data: `$DE74` must be zero and `$DDE8` must be non-zero. `$DDE8` is set by `E`, not by the point selector and not by elapsed time, so a selector followed directly by `RD` is refused however long the program waits. With `E` sent, no additional wait is needed before the entry: the settling time asked for by manual 7.2.2 is a measurement requirement, not a firmware one. The shortest interval actually tested was about a second, that being the controller's own command overhead, against the 2.4 ms manual A1.1.2 allows OPERATE.

The value must lie within 1 % of the range's full scale of the point's nominal. The reference is full scale, not the nominal: a zero point has a nominal of zero and still has a band of 1 % of full scale either side of it. Outside the band the entry is rejected, and the panel shows Err.G19 for an entry made over the bus or Err.P04 for one made from the front panel. Table 9-2 gives the measured edges. `(No query)`

Caution



An `RD` that fails the state test is discarded in **complete silence**. The reject path at `$8D69` skips to the block delimiter and returns without setting an error code, so nothing appears on the panel, nothing appears in the status byte, and the command that follows is accepted normally. A program that omits `E` will run an entire calibration in which not one value is stored.

Note



Entered values accumulate in battery-backed RAM and are not written to the EEPROM until `REND` is sent. A run abandoned before `REND` leaves the stored calibration unchanged, and a power cycle reloads the working copy from the master.

Dispatch $7ABF • handler $8D19 • EXT CAL = ENABLE • Not documented in the Operation Manual.

Every edge in Table 9-2 was found by sweeping until the last accepted and first refused values were adjacent in the seventh digit. The rule holds across the main voltage ranges, the divider ranges, the current ranges and the DAC points, and on both sides of the band.

Table 9-2. Measured Acceptance Edges

Point	Full Scale	Band	Last Accepted	First Refused
1 V range, +F.S	1000 mV	±10 mV	1010.000 mV	1010.001 mV
1 V range, +F.S	1000 mV	±10 mV	990.000 mV	989.999 mV
1 V range, zero	1000 mV	±10 mV	10.00000 mV	10.00001 mV
10 mV divider, zero	10 mV	±0.1 mV	0.100000 mV	0.100001 mV
1 mA range, +F.S	1 mA	±0.01 mA	1.010000 mA	1.010001 mA
DAC points	10 V	±0.1 V	±0.100000 V	0.100001 V

REND

`REND`

CAL END. Commits the calibration to the EEPROM master at `$E000`. The front panel displays a blinking CAL END while the write proceeds. This is the only operation that makes a calibration change persist.

The handler itself clears the point flags and sets a request at `$DE42`; the commit runs from `$1ED7` with interrupts disabled. It has two parts. `$21A2` copies the **entire 364-byte working block** from `$DC29` to the master, one byte at a time with a write-completion poll after each. `$402E` then re-sums what was written and stores the checksum at `$E16C`.

The commit is therefore wholesale, not selective. It does not write only the points entered in the session; it writes whatever the working copy contains at that moment, and certifies the result with a fresh checksum. `(No query)`

Caution



Because the commit copies the whole block and then recomputes the checksum over it, any damage already done to the working copy is made permanent and certified as valid. A coefficient left at zero by an incomplete adjustment, or a byte altered with `MWB`, survives the commit and is reported as sound by both the power-up check and `*TST?`.

Dispatch $798F • handler $7F83 • EXT CAL = ENABLE • Not documented in the Operation Manual.

Memory Access Commands

The two commands in this section read and write the processor's memory space directly. They appear nowhere in the Operation Manual. Both are gated to EXT CAL = DISABLE, so neither can be used to observe the instrument during a calibration session.

MRB

`MRB<address>?`

Reads one byte of the processor's memory space and returns it. <address> is a 16-bit address in hexadecimal, without a dollar sign or other prefix.

This is the route by which the calibration data can be inspected: the working copy at $DC29 and the EEPROM master at $E000 are both readable, 364 bytes each, and comparing them is a cheap check that a commit completed. The response is one byte per query, so a full block requires 364 transactions. `(Query only)`

Dispatch $7AA4 • handler $8BEE • EXT CAL = DISABLE • Not documented in the Operation Manual.

MWB

`MWB<address>,<value>`

Writes one byte to the processor's memory space. <address> is a 16-bit address in hexadecimal and <value> is the byte to store.

The command is accepted and the store instruction at $8C85 executes, but what it achieves depends on the target. A write into the working calibration copy in battery-backed RAM does change the instrument's behaviour, and is lost at the next power-up when the EEPROM master is copied over it. A write into the master's address range has no effect at all: the store executes and the syntax-error bit stays clear, but the value does not change, because an EEPROM requires a programming sequence rather than a processor store. `(No query)`

Caution



There is no restore operation. A calibration image can be read with `MRB` for record or comparison, but it cannot be written back with `MWB`. If a calibration is lost, the values must be re-entered through the normal procedure.

A write into the working copy is only harmless while it stays there. `REND` copies the whole working block to the master, so a poke that would have been discarded at the next power-up becomes permanent if a commit intervenes.

Dispatch $7AAD • handler $8C30 • EXT CAL = DISABLE • Not documented in the Operation Manual.

Output Control Commands

E / OP

`E`

`OP`

OPERATE. Connects the generator to the output terminals. `E` and `OP` are synonyms and reach the same handler; `E` is the form accepted in TR6120A mode.

The handler contains no EXT CAL gate and therefore works with the switch in either position. That is what makes an automated calibration possible: each of the 45 points must be energised before it can be measured. `(No query)`

Caution



On the 100 V and 1000 V ranges the output terminals carry a hazardous voltage while the instrument is in OPERATE.

Dispatch $793E / $7974 • handler $7F5D • EXT CAL = either • manual 4.5.2(1)

H / SB

`H`

`SB`

STANDBY. Disconnects the generator from the output terminals. `H` and `SB` are synonyms and reach the same handler; `H` is the form accepted in TR6120A mode. This is the state the instrument enters at power-up. The handler contains no EXT CAL gate. `(No query)`

Dispatch $796B / $79C5 • handler $821F • EXT CAL = either • manual 4.5.2(1)

V

`V<d>`

Selects a DC voltage range. <d> is 2, 3 or 9 for the 10 mV, 100 mV and 1000 mV divider ranges, or 4, 5, 6 or 7 for the 1 V, 10 V, 100 V and 1000 V main-output ranges. Table 1-1 lists them.

Selecting a divider range routes the output to the separate divider terminals and inhibits the voltage and current limit commands, which then raise Err.G16.

The mnemonic is matched as a prefix and the handler parses the digit itself; the EXT CAL gate is applied in the per-digit branch, and every branch requires DISABLE. The command therefore fails with Err.G15 during a calibration session, which is the error most often seen when the transport characteristic of Chapter 2 truncates `RV` to `V`. `(No query)`

Dispatch $7AFE • handler $835B • EXT CAL = DISABLE, applied per range digit • manual 4.5.2(2)

I

`I<d>`

Selects a DC current range. <d> is 1, 2 or 3 for the 1 mA, 10 mA and 100 mA ranges. As with `V`, the digit is parsed by the handler and the gate is applied per branch. The current ranges are assured within an output compliance of ±10 V. `(No query)`

Dispatch $7A92 • handler $8380 • EXT CAL = DISABLE, applied per range digit • manual 4.5.2(2)

D

`D{+|-}<numeric_value>[<unit>]`

Sets the output value. <numeric_value> carries up to seven characters and may include a decimal point; an excess is truncated. A leading `+` or a space may be given or omitted for a positive value; `-` is required for a negative one.

<unit> selects auto-ranging. **MV** auto-ranges among the 10 mV, 100 mV and 1000 mV divider ranges; **V** auto-ranges among the 1 V, 10 V, 100 V and 1000 V ranges; **MA** auto-ranges among the current ranges. With the unit omitted the range stays as selected and **the number is interpreted in volts**, or in milliamperes on the current ranges, regardless of which range is in force. The Operation Manual's own example is `V6` followed by `D+30` for 30 V on the 100 V range.

In TR6120A mode only the fixed-range form is available; a unit suffix is not accepted. `(No query)`

Note



The `D` unit is not the calibration entry unit. Calibration data for the 1 V range is entered in millivolts, per manual 7.2.2, but `D+1000` on the 1 V range means 1000 V and is rejected with Err.G13. The correct command for full scale on that range is `D+1`.

Dispatch $7A80 • handler $8D95 • EXT CAL = DISABLE • manual 4.5.4

D?

`D?`

Returns the current output setting: a sign, six decimal places and the unit, in the same form as the value field of the `PANE?` record. Measured on the instrument, `D+0.5` on the 1 V range gave `+0.500000 V` and `D-0.25` gave `-0.250000 V`. The value returned is the setting, not a measurement, and is reported whether the output is in OPERATE or STANDBY.

The mnemonic does not appear in the query table of manual 4.6. Its handler is a structural sibling of `PANE?`: both open with the same guard, a test of `$C462` through `$3DAE`, and both select among the same four response builders at `$9627`, `$9630`, `$9639` and `$964C`. `(Query only)`

Dispatch $7935 • handler $859D • EXT CAL = DISABLE • Not documented in the Operation Manual.

VL

`VL<numeric_value>`

Sets the DC voltage limit. <numeric_value> is 10 to 1250, in volts, in steps of 10 V. The value at power-up is 130.

The limit cannot be set while a divider range is selected; the attempt raises Err.G16. A non-divider range must be selected first. The handler is gated to EXT CAL = DISABLE, so the limit can only be changed with the switch in that position, although a limit set beforehand persists across the transition into a calibration session. `(No query)`

Caution



At the power-up setting of VL130 the 1000 V range produces no output at all when it is driven by the `D` command. The failure is silent: range selection succeeds, OPERATE succeeds, and the range's zero point still measures correctly, so only a full-scale measurement reveals it. Any procedure that sets the 1000 V range's output with `D` must raise the limit first, and should restore it afterwards.

Dispatch $7AF5 • handler $7D1C • EXT CAL = DISABLE • manual 4.5.2(14)

IL

`IL<numeric_value>`

Sets the DC current limit. <numeric_value> is 1 to 125, in milliamperes, in steps of 1 mA. The value at power-up is 125, except on the 1000 V range and on the 1 mA and 10 mA ranges of option 01, where it is 13.

As with `VL`, the limit cannot be set while a divider range is selected, and the handler is gated to EXT CAL = DISABLE. `(No query)`

Dispatch $7A89 • handler $7CE0 • EXT CAL = DISABLE • manual 4.5.2(14)

Sense and Guard Commands

SEN0 / SEN1

`SEN0`

`SEN1`

`SEN0` selects internal sense, in which the generator regulates at its own output terminals. `SEN1` selects external sense, in which it regulates at the remote sense terminals.

Neither handler contains an EXT CAL gate, so both work in either switch position. Internal sense is the appropriate setting for a calibration, where the reference standard is connected directly to the output terminals. `(No query)`

Leave it at `SEN0` afterwards. Sense is a backed-up parameter, so unlike the range or the output value it does not reset at the next power-up, and OFF is its initial value; see Table 1-3.

Dispatch $79CE / $79D7 • handler $7E5E / $7E93 • EXT CAL = either • manual 4.5.2(3)

SEN?

`SEN?`

Returns the sense setting as `SEN0` or `SEN1`. Unlike the commands it reports on, the query handler is gated to EXT CAL = DISABLE and is refused during a calibration session. `(Query only)`

Dispatch $79E0 • handler $87EC • EXT CAL = DISABLE • manual 4.6 Table 4-5(1)

GRD0 / GRD1

`GRD0`

`GRD1`

`GRD0` selects internal guard and `GRD1` selects external guard. Neither handler contains an EXT CAL gate. Internal guard is the appropriate setting for a calibration. `(No query)`

Guard is backed up in the same way as sense, and its initial value is also OFF; see Table 1-3.

Dispatch $7950 / $7959 • handler $7E09 / $7E33 • EXT CAL = either • manual 4.5.2(4)

GRD?

`GRD?`

Returns the guard setting as `GRD0` or `GRD1`. The query handler is gated to EXT CAL = DISABLE. `(Query only)`

Dispatch $7962 • handler $869F • EXT CAL = DISABLE • manual 4.6 Table 4-5(2)

Memory, Recall and Program Commands

The instrument holds 100 setup channels, numbered 0 to 99. The commands in this section store into them, recall from them, and run them as a program. None of them is used by a calibration, and all are gated to EXT CAL = DISABLE.

MEM

`MEM<channel>,<range>,<data>[,VL<n>][,IL<n>]`

Stores a complete setup into a memory channel. <channel> is 0 to 99, given as up to two characters. <range> is a range code in the form accepted by `V` or `I`, and <data> is an output value in the form accepted by `D`, with the restriction that a unit suffix may not be used when the range is fixed.

The voltage and current limit fields are optional. If they are omitted the channel takes the default limits, VL130 and IL125, or IL13 on the 1000 V range and on the 1 mA and 10 mA ranges of option 01. Limits may not be given at all for the divider ranges V2, V3 and V9; an attempt raises Err.G16 and the whole entry is discarded.

The two limit fields are not independent. Giving `IL` without `VL` raises Err.G18; giving `VL` without `IL` raises Err.G01. Either both are present, in that order, or neither is.

If any field is in error the entry is ignored and all subsequent codes in the block are ignored with it. `(No query — see MEM?)`

Note



The voltage limit is clamped, not discarded. A value above 1250 is rejected outright, and one whose low digit is not zero is rejected with it. What survives is then clamped at `$9061`: unless the range code is 0, or 8 or 9 with option 01 fitted, a voltage limit greater than 130 is silently reduced to 130. Since range code 0 is the 1000 V range, every other range accepts at most VL0130. Measured on the 1 V range: `VL0100` stored as 100, `VL0130` as 130, and `VL0140` as 130. The current limit is clamped the same way in mirror image at `$90AD`, to 13 mA, and only on range code 0 — which is the origin of the IL13 default the Operation Manual quotes for the 1000 V range.

Dispatch $7A9B • handler $8B55 • EXT CAL = DISABLE • manual 4.5.3

MEM?

`MEM<channel>?`

`MEM<first>,<last>?`

Returns the contents of a memory channel. The single-channel form takes one channel number; the range form takes a first and a last and returns one record per channel, the records separated by semicolons.

A record has the form

```
MEM00,V4, +0.000000 V,VL0130,IL125
```

— the channel number as two digits, the range code, the output value with its unit, the voltage limit as four digits and the current limit. This is the factory default record; a channel that has never been written returns exactly this. `(Query only)`

Note



The range form returns up to a hundred records in one response and is best avoided over a slow controller; querying one channel at a time is markedly more reliable. A limit read back lower than the one written is the clamp described under `MEM`, not a failure of the store.

EXT CAL = DISABLE • manual 4.6

Where a Channel Lives

The hundred setups are not held as records. Each field has its own array, indexed by channel, and the store at `$9174` writes all four in turn. Table 9-3 gives the layout, which is also what `MEM?` reads back through the accessors at `$99DB` to `$9A1B`.

Table 9-3. Setup Memory Arrays

Field	Base	Stride	Accessor	Written By
Range code	$D9BA	1 byte	$99DB	$9181
Sign	$DBAE	1 byte	$99E7	$918B
Output value	$D69A	8 bytes	$99F3	$9196
Voltage limit	$DA1E	2 bytes	$9A0A	$91A3
Current limit	$DAE6	2 bytes	$9A1B	$91B0

The arrays are filled with the defaults at `$57A0` and `$5830`, which is where VL0130 and IL125 come from. Those two routines and the store at `$9174` are the only writers, so a channel that has never been written reads back the initialisation value and nothing else can disturb it.

RCL

`RCL<channel>`

Enters program mode and positions the program at the given channel. <channel> is 0 to 99. A channel number of more than two digits raises Err.P05. `(No query)`

Caution



The instrument stops answering queries once this command has been accepted, because it has changed mode — not, as earlier editions of this document said, because it is unusually susceptible to the transport characteristic of Chapter 2. **Send `D` with a value to return to direct mode**; that is the documented escape and it was measured to restore `$DDED` to 0. A device clear also works and additionally stops any running program.

Note



The Operation Manual describes this as a recall, and it is not one. The handler sets `$DDED` at `$8D03`, which is the program-mode flag; it does not apply the stored setup. Read from the instrument, `$DDED` is 0 before `RCL00` and 1 after. The setup is applied when the program runs, which is why `D?` and `PANE?` continue to report the settings already in force. To reproduce a stored setup in direct mode, read it back with `MEM?` and re-issue the fields explicitly.

Dispatch $7AB6 • handler $8C8D • EXT CAL = DISABLE • manual 4.5.2(5)

ST

`ST{0|1|2}`

Selects the recall mode used when a scan is run. `ST0` selects single scan, `ST1` repeat scan and `ST2` step scan. The digit is parsed by the handler and stored at `$C0D2`, but not as written: `ST0` stores 1, `ST1` stores 2 and `ST2` stores 0. Several other handlers branch on that byte, so the correspondence matters when reading the firmware.

Note



Sent while a program is running, this command raises Err.G20 directly at `$801F` and the mode is not changed. It is the only command that raises G20 without also testing `$C0D2`.

Dispatch $7AEC • handler $7FEC • EXT CAL = DISABLE • manual 4.5.2(6)

ST?

`ST?`

Returns the recall mode as `ST0`, `ST1` or `ST2`. `(Query only)`

Dispatch $7A04 • handler $87C2 • EXT CAL = DISABLE • manual 4.6 Table 4-5(5)

STM

`STM<numeric_value>`

Sets the step time used between channels during a scan. <numeric_value> is 1 to 99 seconds. `(No query)`

Dispatch $7AE3 • handler $8277 • EXT CAL = DISABLE • manual 4.5.2(5)

STM?

`STM?`

Returns the step time in the form `STMx`, where x is two digits. `(Query only)`

Dispatch $7A0D • handler $866E • EXT CAL = DISABLE • manual 4.6 Table 4-5(3)

SC

`SC<first>,<last>`

Sets the first and last channels of the scan. Both are 0 to 99, and the first must not exceed the last; an attempt to set them the other way round raises Err.P03. `(No query)`

Dispatch $7ADA • handler $7C1E • EXT CAL = DISABLE • manual 4.5.2(5)

SC?

`SC?`

Returns the scan limits in the form `SCx,y`, each of two digits, leading zeros included. `(Query only)`

Dispatch $7947 • handler $861F • EXT CAL = DISABLE • manual 4.6 Table 4-5(4)

STT / *TRG

`STT`

`*TRG`

Starts the program, or continues it after a pause. The two mnemonics reach the same handler. Both are effective while a scan is running in program mode. `(No query)`

Dispatch $7A1F / $7A5E • handler $7C97 • EXT CAL = DISABLE • manual 4.5.2(7)

PAU

`PAU`

Pauses a running program. `STT` resumes it. `(No query)`

Dispatch $7986 • handler $7CBC • EXT CAL = DISABLE • manual 4.5.2(7)

STP

`STP`

Stops a running program and returns it to the first channel. `(No query)`

Dispatch $7A16 • handler $7C80 • EXT CAL = DISABLE • manual 4.5.2(7)

Status and Service Request Commands

*CLS

`*CLS`

Clears the status byte to all zeros. It does not alter the mask set by `SMS`. `(No query)`

Dispatch $7A31 • handler $7BC1 • EXT CAL = DISABLE • manual 4.5.2(10)

*STB?

`*STB?`

Returns the status byte as a single numeric value, 0 to 255. The bit assignments are given in manual 4.7.1 for R6161 mode and 4.7.2 for TR6120A mode. `(Query only)`

Note



`*STB?` is itself subject to the transport characteristic of Chapter 2. Truncated to `STB?` it reaches the `ST` handler, which rejects **B** as a mode digit and raises Err.G11 — setting the very bit the query is being used to read.

Dispatch $7A55 • handler $8920 • EXT CAL = DISABLE • manual 4.6 Table 4-5(10)

SMS

`SMS<numeric_value>`

Sets the mask register applied to the status byte. <numeric_value> is 0 to 255; a value of 255 enables every bit. The mask is equivalent to the serial-poll enable register.

The mask powers up at 255, with every bit enabled, so command errors are visible to a polling program without any change. Setting the mask to 0 is what suppresses them: with `SMS0` in force an invalid command leaves the status byte reading zero. `(No query)`

Dispatch $7B07 • handler $82A4 • EXT CAL = DISABLE • manual 4.5.2(11)

SMS?

`SMS?`

Returns the mask register as a single numeric value, 0 to 255. `(Query only)`

Dispatch $7A4C • handler $88D0 • EXT CAL = DISABLE • manual 4.6 Table 4-5(11)

SRQON / S0

`SRQON`

`S0`

Enables the service request. `S0` is the short form used in TR6120A mode; both reach the same handler. `(No query)`

Dispatch $79F2 / $79B3 • handler $824D • EXT CAL = DISABLE • manual 4.5.2(9)

SRQOF / S1

SRQOF

S1

Disables the service request. S1 is the short form used in TR6120A mode; both reach the same handler. This is the state at power-up in TR6120A mode. (No query)

Dispatch $79E9 / $79BC • handler $822F • EXT CAL = DISABLE • manual 4.5.2(9)

SRQ?

SRQ?

Returns the service request setting as SRQON or SRQOF. (Query only)

Dispatch $79FB • handler $88F0 • EXT CAL = DISABLE • manual 4.6 Table 4-5(8)

*TST?

*TST?

Runs the self-diagnostic test and returns the result as a single numeric value, 0 to 15. Bit 0 is a ROM sum error, bit 1 a RAM read error, bit 2 a RAM parameter error, bit 3 an EEPROM data error and bit 4 a fan stop error. Bits 5 to 7 are always zero.

Bit 3 is the bit to watch after a calibration: it reports that the EEPROM contents failed their integrity check, in which case the instrument substitutes the ROM defaults at $9AE9 and operates out of adjustment. (Query only)

Dispatch $7A67 • handler $880F • EXT CAL = DISABLE • manual 4.6 Table 4-5(9)

System Commands

*IDN?

`*IDN?`

Returns the manufacturer, model and firmware revision, from the string at $9DE3. The instrument examined returns ADVANTEST,R6161,REV A05; manual Table 4-5 gives REV A01 as its example response, the revision field tracking the firmware rather than the manual printing.

All three fields are usable as instrument identity on a calibration certificate, and the tool takes them from here rather than asking for them. What the response does *not* carry is a serial number or an option list: the response string is a fixed constant in ROM, identical in every R6161, so neither can be read over the bus and both have to be taken from the rear-panel label. This predates the four-field manufacturer/model/serial/revision convention that later SCPI instruments follow.

Because a truncated `*IDN?` cannot answer, a no-retry `*IDN?` is an unambiguous indicator of whether the transport characteristic of Chapter 2 is in effect, and it is the query used for the measurements in Tables 2-1 to 2-3. `(Query only)`

Dispatch $7A3A • handler $86C2 • EXT CAL = DISABLE • manual 4.6 Table 4-5(6)

PANE?

`PANE?`

Batch read of the current parameter data. Returns the range code, the output value with polarity and unit, and the voltage and current limits, as a comma-separated record. The field formats are given in manual 4.6.1: the range code is two characters, the output value up to twelve, and each limit is a header followed by fixed-width digits with leading zeros retained. `(Query only)`

Caution



This query does not always return data. When the flag at `$C00B` is set the handler branches at `$8729` and returns the twelve-character ASCII string `Error.FAN !!` instead. A program that parses the reply as a comma-separated record will fail on it, and the failure will look like a communication fault rather than an instrument fault. The same substitution happens on `D?`.

Dispatch $797D • handler $86DC • EXT CAL = DISABLE • manual 4.6.1

DL

`DL{0|1|2|3}`

`DL?`

Selects the block delimiter the instrument appends to its responses. `DL0` selects CR LF with EOI, `DL1` LF only, `DL2` EOI only and `DL3` LF with EOI. `DL0` is the setting established by `C`.

The query form returns the setting in the same notation. No separate dispatch entry exists for `DL?`; the mnemonic `DL` is matched and the handler parses the question mark itself.

None of the four settings avoids the transport characteristic of Chapter 2. All were measured, together with EOI suppression, and are reported in Table 2-4.

Dispatch $7A77 • handler $7BD9 • EXT CAL = DISABLE • manual 4.5.2(8), 4.6 Table 4-5(7)

C

`C`

Initialises the GPIB setup, turns the output off and stops all operations. In direct operation mode the instrument is left at zero on the 1 V range; in program operation mode the program is stopped and returned to its first channel. The effect is the same as that of the GPIB Selected Device Clear and Device Clear messages.

The columns of manual Table 4-3 headed "initial value set by program code C" record what each setting becomes. `C` does not affect the calibration data. `(No query)`

Dispatch $792C • handler $7BA1 • EXT CAL = DISABLE • manual 4.5.2(12), 4.5.6(5)

Z / *RST

`Z`

`*RST`

Initialises the instrument. Both mnemonics reach the same handler. Manual 4.5.2(13) states that the battery-backed program data and each parameter are initialised, and that the processing of `C`, SDC and DCL is included.

Dispatch $7A28 / $7A43 • handler $7FAE • EXT CAL = DISABLE • manual 4.5.2(13)

Warning



Never send `Z` or `*RST`. The calibration coefficients are among the battery-backed data these commands initialise, and there is no undo: the working copy cannot be written back over the bus, so a lost calibration must be re-entered through the whole 45-point procedure. Neither command should be used as a convenient invalid command when testing a controller.

TR6120A Mode Commands

The instrument can be placed in a compatibility mode in which it responds to the shorter command set of the earlier TR6120A. The firmware carries a second, shorter comparison chain for it. `E`, `H`, `D`, `S0`, `S1` and `C` are shared with the R6161-mode chain and are described above; the entries below are reached only in TR6120A mode.

The mode is selected from the **front panel**, not from a rear-panel switch and not over the bus. Manual 4.4 gives the procedure: press LOCAL to enter GPIB Addressing mode, then press CONTINUOUS key ⑤ repeatedly to toggle the leftmost display field between `b1` (R6161 mode) and `20` (TR6120A mode), and press ENTER to commit. Further presses of LOCAL step on through BCD Remote Setup mode to Normal operation mode. Because the selection is a panel operation, a program has no way to change or interrogate it; the mode in force must be established by inspection before a session begins.

V4 / V5 / V6 / V7

`V4`
`V5`
`V6`
`V7`

Select the 1 V, 10 V, 100 V and 1000 V ranges. Unlike the R6161-mode `V` command, each range has its own dispatch entry and its own handler. The divider ranges are not available in TR6120A mode. `(No query)`

Dispatch $7B64 / $7B6D / $7B76 / $7B7F • handlers $83E8 / $8412 / $843C / $8466 • EXT CAL = DISABLE • manual 4.5.6(2)

I1 / I2 / I3

`I1`
`I2`
`I3`

Select the 1 mA, 10 mA and 100 mA ranges, each with its own dispatch entry and handler. `(No query)`

Dispatch $7B2E / $7B37 / $7B40 • handlers $84B9 / $84E2 / $850B • EXT CAL = DISABLE • manual 4.5.6(2)

L

`L{1|2|3|4|5|6|7|8}`

Sets the output limit in TR6120A mode. The digit selects one of eight fixed values rather than a numeric limit: `L1` 10 V, `L2` 130 V, `L3` 300 V and `L4` 1250 V for the voltage limit; `L5` 5 mA, `L6` 13 mA, `L7` 50 mA and `L8` 125 mA for the current limit. `L2` and `L8` are the settings at power-up.

The handler contains no EXT CAL gate and so works in either switch position, which distinguishes it from the R6161-mode `VL` and `IL` commands. `(No query)`

Dispatch $7B49 • handler $7EC9 • EXT CAL = either • manual 4.5.6(3)